Optimized 5G Rollout Planning Using Geo-Spatial Data Analysis

Moohammed Sattar Basheer Ahamed

Student No: R00259436

Supervisor: **Dave Clarke**

For the module DATA9003 – Research Project as part of the

Master of Science in Data Science and Analytics, Department of Mathematics, 20/08/2025

Declaration of Authorship

I, Moohammed Sattar, declare that this thesis titled ‘Optimized 5G Rollout Planning Using Geo-Spatial Data Analysis’ and the work presented in it are my own. I confirm that,

- This work was done wholly or mainly while in candidature for the Masters’ degree at Munster Technological University
- Where any part of this thesis has previously been submitted for a degree or any other qualification at Munster Technological University or any other institution, this has been clearly stated
- Where I have consulted the published work of others, this is always clearly attributed
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this project report is entirely my own work
- I have acknowledged all main sources of help
- I understand that my project documentation may be stored in the library at MTU, and may be referenced by others in the future

Signed: Moohammed Sattar B

Date: 20/08/2025

Acknowledgment

I would like to express my deepest gratitude to my supervisor, Dave Clarke, for his constant guidance, support, and encouragement throughout this project. whose insights and feedback have been invaluable in shaping this work.

I also extend my thanks to all professors and staff at the university whose teaching and advice have given me the knowledge and skills needed to complete this thesis. Their dedication to students has been a true source of inspiration.

Finally, I would like to acknowledge the wider data science community, whose shared knowledge, open-source tools, and research have provided the foundation and resources that made this work possible.

Table of Contents

Abstract 1
1.Introduction 2
1.1 Background 2
1.2 Problem Statement 3
1.3 Research Aim 3
1.4 Research Objectives 4
1.5 Scope of the Study 4
1.6 Contribution of the Study 5
1.7 Structure of the Thesis 5
2. Literature Review 6
2.1 Introduction 6
2.2 Propagation Models for Coverage Estimation 6
2.2.1 Critical Insights 7
2.3 Infrastructure Deployment and Evaluation 8
2.3.1 Critical Insights 8
2.4 Clustering Approaches for Hotspot Detection 9
2.5 Terrain and Elevation Considerations 10
2.6 Optimization Approaches 10
2.6.1 Greedy and Heuristic Methods 10
2.6.2 Redundancy Pruning 10
2.6.3 Critical Insights 11
2.7 Gaps in Literature 11
2.8 Synthesis of Literature 11
3. Methodology 13
3.1 Introduction 13
3.2 Data Sources and Preprocessing 13
3.2.1 Population Data 13
3.2.2 Water Bodies and Filtering 14
3.2.3 Elevation Data 15
3.2.4 Existing Towers 16
3.2.5 Roads 16
3.3 Clustering of Demand Hotspots 17

3.3.1 Rationale 17
3.3.2 DBSCAN 17
3.3.3 HDBSCAN 18
3.3.4 Implementation 18
3.4 Elevation-Aware Candidate Ranking 18
3.4.1 Terrain Metrics 18
3.4.2 Population Proximity 20
3.4.3 Final Candidate Score 20
3.5 Coverage and Capacity Modeling 20
3.5.1 Path Loss Models 20
3.5.2 Frequency Bands 20
3.5.3 Coverage Thresholds 21
3.5.4 Capacity Estimation 21
3.6 Optimization and Site Selection 22
3.6.1 Must-Serve Coverage 22
3.6.2 Redundancy Pruning 22
3.6.3 Capacity Overlay and Small Cells 22
3.7 Visualization 23
3.8 Summary of Methodology 23
4. Results 25
4.1 Introduction to Results 25
4.2 Data Preprocessing Outcomes 25
4.2.1 Population Data Cleaning 26
4.2.2 Removal of Water Body Interference 26
4.2.3 Elevation Data Integration 27
4.2.4 Existing Tower Integration 27
4.2.5 Demand Distribution Summary 28
4.2.6 Visualization of Pre-processed Data 28
4.2.7 Summary 28
4.3 Clustering Results 29
4.3.1 Choice of Clustering Method 29
4.3.2 Implementation of HDBSCAN 30
4.3.3 Comparison with DBSCAN 30

4.3.4 Characteristics of Clusters 31
4.3.5 Cluster Medoid Selection 31
4.3.6 Visualization of Clustering Results 32
4.3.7 Summary 32
4.4 Elevation-Aware Candidate Site Ranking 32
4.4.1 Rationale for Terrain Integration 33
4.4.2 Terrain Metrics 33
4.4.3 Composite Scoring Formula 33
4.4.4 Ranking Results in Cork 34
4.4.5 Comparison with Literature 34
4.4.6 Summary 35
4.5 Coverage Modeling and Capacity Estimation 35
4.5.1 Rationale for Coverage Modeling 35
4.5.2 COST-231 Hata Model 36
4.5.3 Coverage by Frequency Bands 36
4.5.4 Capacity Estimation with Shannon Formula 37
4.5.5 Utilization Metric 37
4.5.6 Results for Cork 38
4.5.7 Comparative Analysis with Literature 39
4.5.8 Summary 39
4.5.9 Interpretation of Negative SNR Values 39
4.6 Optimization Process Results 41
4.6.1 Rationale for Optimization 41
4.6.2 Stage 1: Must-Serve Coverage (Set-Cover) 41
4.6.3 Stage 2: Redundancy Pruning 42
4.6.4 Stage 3: Capacity Overlays (NR3500 Macros) 42
4.6.5 Stage 4: Small Cell Hotspots 43
4.6.6 Results of Optimization 43
4.6.7 Visualization of Optimization 44
4.6.8 Comparative Insights 44
4.6.9 Summary 44
4.7 Visualization Outputs 45
4.7.1 Visualization of Pre-processed Data 45

4.7.2 Visualization of Clustering Results 46
4.7.3 Visualization of Coverage Modeling 46
4.7.4 Visualization of Optimization Outcomes 49
4.7.5 Communication Value 50
4.7.6 Summary 51
5.Discussion and Conclusion 52
5.1 Reflection on Pipeline Stages 52
5.2 Limitations 54
5.3 Implications for Policy and Practice 54
5.4 Future Research Directions 55
5.5 Practical Implications for Startups and Operators 55
5.6 Conclusion 55
References 57
Appendix 60
1.Data Preprocessing 60
1.1 Parameters and constants initialization 60
1.2 Helper Functions 61
1.3 Population Data 62
1.4 Elevation Data 62
1.5 Water Polygon data 63
1.6 Loading the existing tower data 63
1.7 Road Network (OSMnx): edges to GPKG (layer='edges') 63
2. 5G Rollout Planning 64
2.1 Parameters and constants initialization 64
2.2 Must-serve population 65
2.3 RF Helper functions 65
2.4 Demand Clustering with DBSCAN and HDBSCAN 66
2.5 Elevation aware Macro ranking 67
2.6 Macro coverage calculation 69
2.7 Filter to get a realistic Macro subset 70
2.8 Enriching the selected Macros for better coverage 72
2.9 BackHaul demand calculation 73
2.10 Overlay Tuning 73

2.11 Tower candidate selection 74
2.12 Greedy selection of candidates that can handle the required capacity 75
2.13 Recomputing NR/Small-cell per-site metrics after selection 77
2.14 Assembling all the selected sites 79
2.15 Elevation data retrieval for selected sites 79
2.16 Calculating the final Capacity and RF parameters for cells 80
2.17 Backfill coverage/population calculation 82

List of Figures

Figure 1.1.1: 5G bandwidths and their coverage ……2

Figure 2.2.1: Evolution of Propagation Models……..7

Figure 2.4.1: DBSCAN vs HDBSCAN……..9

Figure 2.8.1: Prior approaches feeding into this study……..12

Figure 3.2.1: Cleaned Population dataset……..14

Figure 3.2.2: Elevation data……..15

Figure 3.2.3: Existing Towers in Cork……..16

Figure 3.2.4: Road Network of Cork……17

Figure 3.3.1 Terrain Metrics……19

Figure 3.6.1: Optimization Workflow……...22

Figure 3.8.1: Pipeline Workflow for 5G Rollout Planning……23

Figure 4.7.1 Heatmap of cleaned population data……45

Figure 4.7.2 NR Overlays……47

Figure 4.7.3 Existing Macro cells selected for reuse……47

Figure 4.7.4 New Macros for deployment……48

Figure 4.7.5 New Small Cells covering high demand spots……48

Figure 4.7.6: Final Tower candidates……49

Figure 4.7.7: Interactive map with detailed view of a Small Cell ……50

Figure 4.7.8: Interactive map with detailed view of a Macro……50

List of Tables

Table 2.2.1: Summary Table of Models …….7

Table 2.4.1 Summary Table of Clustering Methods…….10

Table 4.2.1: Summary of Population Demand Distribution after Preprocessing…….28

Table 4.3.1: Population Distribution across HDBSCAN Clusters……31

Table 4.4.1: Top 10 Candidate Sites Ranked by Population and Terrain Score……34

Table 4.6: Coverage footprint per frequency band……37

Table 4.7: Must-serve coverage diagnostics…….38

Table 4.8: Effect of overlays and small cells on utilization……...43

List of Abbreviations

1. **5G** – Fifth Generation Mobile Network
2. **4G** – Fourth Generation Mobile Network
3. **LTE** – Long Term Evolution
4. **NR** – New Radio (5G standard)
5. **LTE700** – LTE operating at 700 MHz band
6. **LTE800** – LTE operating at 800 MHz band
7. **LTE1800** – LTE operating at 1800 MHz band
8. **NR3500** – NR operating at 3500 MHz band
9. **SNR** – Signal-to-Noise Ratio
10. **PL** – Path Loss
11. **LOS** – Line of Sight
12. **NLOS** – Non-Line of Sight
13. **COST-231** – Cooperative for Scientific and Technical Research 231 (Hata model extension)
14. **ITU** – International Telecommunication Union
15. **AOI** – Area of Interest
16. **CRS** – Coordinate Reference System
17. **EPSG** – European Petroleum Survey Group (spatial reference codes)
18. **GIS** – Geographic Information System
19. **DEM** – Digital Elevation Model
20. **DSM** – Digital Surface Model
21. **DTM** – Digital Terrain Model
22. **GDF** – GeoDataFrame (GeoPandas object)
23. **OSM** – OpenStreetMap
24. **CSV** – Comma-Separated Values
25. **JSON** – JavaScript Object Notation
26. **API** – Application Programming Interface
27. **PDF** – Portable Document Format
28. **ML** – Machine Learning
29. **AI** – Artificial Intelligence
30. **RF** – Random Forest

31. **CNN** – Convolutional Neural Network
32. **LR** – Logistic Regression
33. **NB** – Naive Bayes
34. **GBM** – Gradient Boosting Machine
35. **HDBSCAN** – Hierarchical Density-Based Spatial Clustering of Applications with Noise
36. **DBSCAN** – Density-Based Spatial Clustering of Applications with Noise
37. **KNN** – K-Nearest Neighbors
38. **CPU** – Central Processing Unit
39. **GPU** – Graphics Processing Unit
40. **POP** – Population (dataset field)
41. **UTIL** – Utilization (tower usage indicator)

Abstract

The implementation of 5G networks represents a crucial process to satisfy increasing demands for fast and trustworthy connectivity. The deployment of new towers comes with high expenses while unplanned installation produces insufficient rural coverage together with excessive network duplication in densely populated areas. In this study, a data-driven approach was used to guide the placement of towers in County Cork, Ireland, through the integration of population patterns with terrain information, clustering and RF wave modeling.

The initial task involved data cleaning and preparation steps. Coordinates from all datasets were standardized into a consistent geographic projection while the program excluded water sections to limit tower installations to available ground. The analysis integrated elevation data obtained from digital terrain models to analyze how terrain elevation affects signal propagation. The HDBSCAN clustering method identified high-demand regions because it automatically detects different-shaped clusters without requiring pre-defined cluster numbers.

The system integrated candidate tower locations with existing tower sites after identifying them through cluster processing. The path loss models calculated coverage while elevation-aware ranking selected sites located on advantageous terrain features such as ridges. The optimization process used a greedy algorithm to protect and serve all cells with high population density while eliminating sites that added minimal coverage to reduce resource cost.

The results showed that integrating terrain and clustering based analysis produced better coverage compared to simple grid-based methods. The elevation-based optimization approach delivered enhanced rural service coverage while the removal of redundant sites enhanced the overall system utilization. The final tower network distribution became more balanced throughout County Cork when displayed on interactive maps built with Folium.

1.Introduction

1.1 Background

The development of wireless communication technology transformed information access and service delivery through successive generations which brought faster and more portable solutions. The fifth-generation systems operate at bandwidth levels that exceed 4G by many times while achieving below 10 millisecond latency and supporting millions of devices per square kilometre. The new technologies including real-time remote monitoring and autonomous vehicles and augmented reality and industrial automation would be impossible without these features.

The 4G network operated mainly through low- and mid-band spectrum but 5G networks utilized a combination of low, mid and high frequency bands. The different frequency bands possessed distinct advantages and disadvantages. The 700 MHz band provided extensive coverage with good building penetration yet delivered reduced network capacity. The 3.5 GHz mid-band spectrum delivered better capacity than other bands but its transmission distance was shorter and it suffered more interference from obstacles. The high bands above 24 GHz provided maximum throughput but their range was restricted to short distances and they required direct line-of-sight connections.

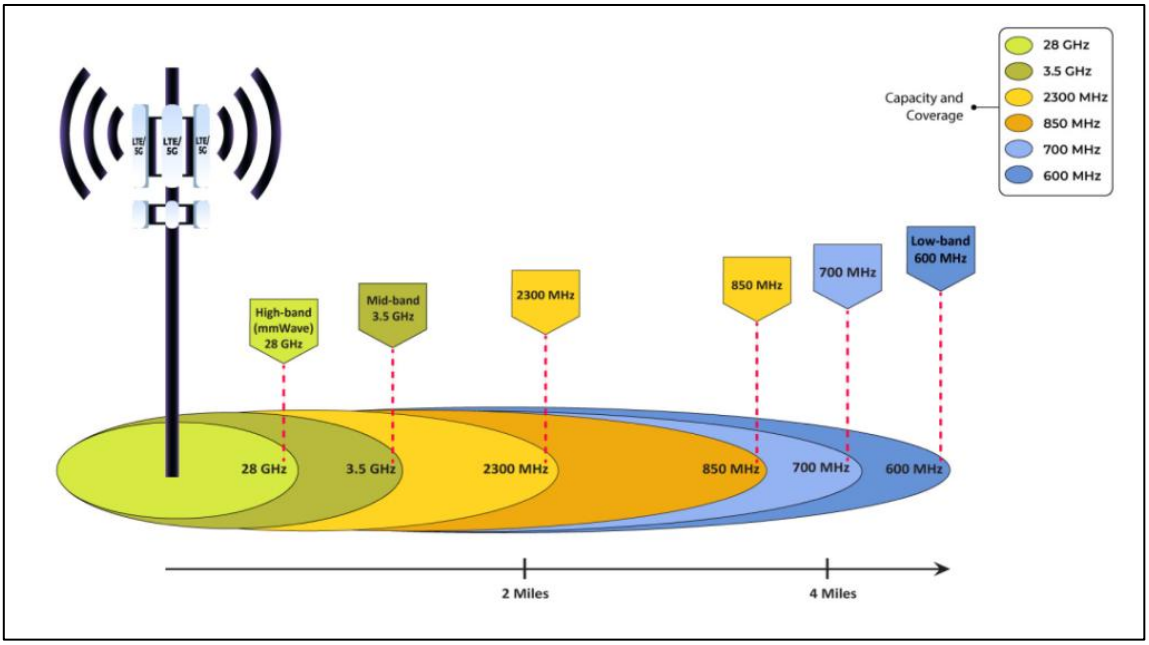

Figure 1.1.1: 5G bandwidths and their coverage (Source: “https://nybsys.com/5g-bands/”)

The combination of different spectrum bands presented a planning difficulty. Operators needed to develop precise strategies for placing new towers to achieve reliable coverage. The distance between towers became a problem when it exceeded a certain limit because it created empty areas particularly in rural regions. When towers were positioned too near each other it resulted in unnecessary resource consumption while creating duplicate coverage areas. The efficient deployment needed spatial data integration with RF wave propagation models and demand analysis for successful implementation.

1.2 Problem Statement

The implementation of 5G networks demanded substantial financial resources to develop both infrastructure and planning systems. The establishment of a new macro site required investments exceeding hundreds of thousands of euros while dense areas needed small cells and overlays for coverage. The improper planning of 5G networks would create two major problems: rural areas would lose their coverage and urban areas would experience redundant infrastructure [7].

County Cork, Ireland, served as an optimal testing ground for evaluating optimized rollout strategies. County Cork, Ireland, presented a wide range of landscapes because it spanned from the densely populated urban area of Cork City to agricultural fields and coastal regions. The varying population density across the area demanded both extensive low-band coverage for wide areas and high-capacity solutions for densely populated regions. Radio propagation became more complex because of terrain elevation which caused hills and slopes to distort signals [8].

The main objective of this study involved creating a data-driven approach to tower placement which would minimize rural coverage gaps while preventing unnecessary duplication in densely populated areas. The solution needed a structured pipeline that combined population data with elevation information and clustering methods and radio wave propagation models.

1.3 Research Aim

The goal of this study was to build a reproducible and data-driven pipeline for optimized 5G rollout planning in County Cork, that could be later deployed for any area in the world. The plan was to build an approach using combination of spatial analysis, clustering algorithms, and path loss modeling to identify and refine candidate tower sites. The expected outcome was

complete and improved coverage and utilization compared to grid-based or other native methods.

1.4 Research Objectives

To achieve the research aim, the study was guided by the following objectives:

1. To pre-process population and spatial datasets into consistent geographic schema for easy access.
2. To clean data by removing missing values and filtering out unusable regions such as water.
3. To integrate elevation data from digital elevation models to account for terrain effects.
4. To identify demand hotspots using clustering methods such as HDBSCAN, which can detect clusters of varying densities without requiring a fixed number [10]– [13].
5. To generate candidate tower locations by combining hotspot clusters with existing infrastructure.
6. To apply path loss models, particularly COST-231 and its variations, to estimate coverage at different frequencies [1], [3], [5], [6].
7. To optimize tower selection through a greedy-approach that ensured must-serve populations were covered, while redundant towers were pruned.
8. To evaluate network efficiency through utilization analysis and population coverage.
9. To visualize candidate and final tower placements using interactive mapping tools.

1.5 Scope of the Study

The study focused exclusively on County Cork which is located in Ireland. Population statistics served as traffic demand indicators because operator statistics were unavailable. Digital terrain models provided elevation data which simulated how hills and slopes affect propagation. The analysis focused on LTE700, LTE800, LTE1800, and NR3500 bands because these frequencies represented low- and mid-band coverage options for 5G deployment [7].

The study did not include millimeter-wave frequencies or economic cost modeling within its scope. The analysis did not include backhaul fiber infrastructure. The project focused mainly on geospatial analysis and radio coverage assessment instead of performing a complete economic deployment simulation.

1.6 Contribution of the Study

The study established a systematic method for 5G tower placement through the integration of clustering and elevation-aware modeling and optimization techniques. The importance of precise path loss prediction emerged from previous research studies [1], [3]– [6] and antenna parameters [2], [9]. Research demonstrated that DBSCAN and HDBSCAN clustering methods proved successful for identifying spatial data hotspots [10]– [13]. The study built upon existing research to establish that:

1. Elevation-aware site ranking improved tower placement compared to grid-based baselines.
2. HDBSCAN clustering detected population concentrations more effectively than uniform hexagonal grids.
3. Pruning reduced redundancy and improved utilization.
4. The Folium maps used for tower placement visualization allowed users to easily verify the optimized rollout through intuitive map-based assessment.

The study made dual practical and methodological contributions by offering a repeatable workflow that could be modified for different geographic areas and spectrum bands.

1.7 Structure of the Thesis

The remainder of this thesis was organized as follows:

1. **Chapter 2 Literature Review**: examined related studies on 5G rollout planning, clustering algorithms, and wave propagation models.
2. **Chapter 3 Methodology**: described the datasets, preprocessing steps, clustering, and path loss models used.
3. **Chapter 4 Results**: presented the outcomes of coverage, capacity, and optimization, with supporting figures and maps.
4. **Chapter 5 Discussion and Conclusion**: summarized findings, highlighted contributions, outlined limitations, and suggested future work such as scaling to larger regions and integrating frequency-specific propagation.2. Literature Review

2. Literature Review

2.1 Introduction

The deployment of 5G networks received extensive research because users demanded enhanced capacity and broader coverage and dependable performance. Research into three core subjects included radio propagation modeling and coverage predictions as well as tower deployment planning and infrastructure development and clustering methods with machine learning approaches for hotspot detection. This review systematized the research by category to help establish an optimized deployment process in County Cork.

2.2 Propagation Models for Coverage Estimation

The accurate modeling of propagation waves proved vital for tower planning because it enabled signal strength prediction along with coverage area estimation. The cellular planning industry adopted the Okumura Hata and COST-231 models as their main propagation models [3]. Correia [1] explained the COST 231-Bertoni-Ikegami model which enhanced urban and suburban coverage predictions through its ability to model building effects and diffraction. Sun et al. [4] analyzed the prediction accuracy of large-scale path loss models for 5G communication systems by demonstrating that stable parameters produce reliable coverage estimates.

Multiple research efforts modified the COST-231 model for better performance in specific locations. Akanni et al. [5] enhanced the COST-231 Hata model for tropical regions operating at 1800 MHz while Verma and Saini [6] used statistical methods to adjust the model for dense urban environments. Singh [3] performed model comparisons among Okumura Hata and COST-231 which demonstrated that each model performed best under specific frequency and environmental conditions. Research shows that coverage prediction requires knowledge about both the transmission frequency and local terrain characteristics.

Model	Frequency Range	Key Features	Limitations	Sources
Okumura-Hata	150–1500 MHz	Empirical, simple	Not reliable above 2 GHz	[3]
COST-231 Hata	up to 2 GHz	Urban/suburban extensions	Needs tuning per region	[1], [5], [6]
COST-231 W-I	up to 2 GHz	Considers diffraction, buildings	Complex parameterization	[1], [4], [6]
Tuned versions	Region-specific	Better accuracy	Lacks generalization	[5], [6], [7]

Table 2.2.1: Summary Table of Models

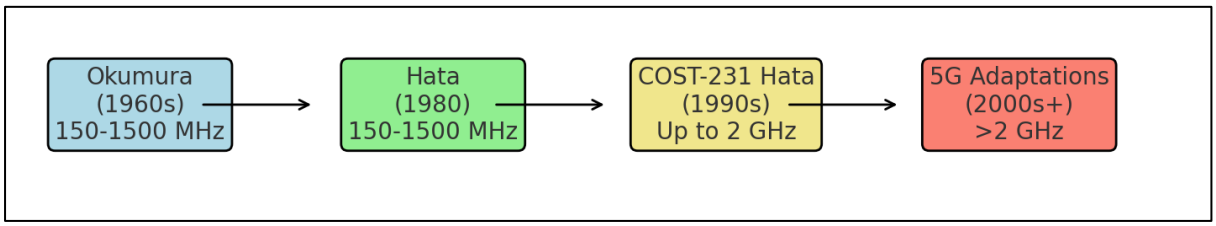

Figure 2.2.1: Evolution of Propagation Models

Saakian [8] presented an extensive theoretical explanation about radio wave propagation which included line-of-sight and diffraction and reflection phenomena. Basic concepts from these studies enabled the combination of terrain-sensitive methods. The study by Xu et al. [9] evaluated tower transmission antenna configurations to demonstrate how tower height combined with antenna placement determines both electromagnetic safety measures and network coverage quality. The study demonstrated that elevation data and antenna height need to be integrated into planning models.

2.2.1 Critical Insights

From these studies, three gaps emerged:

1. Most models required region-specific calibration.
2. Terrain and elevation effects were not fully captured by traditional models.

3. High-frequency (3.5 GHz and beyond) modeling remained less explored compared to LTE bands.

These gaps justified integrating digital elevation models into this project's pipeline.

2.3 Infrastructure Deployment and Evaluation

The studies of infrastructure deployment focused on evaluating their deployments based on coverage and throughput performance. The research by Affandi et al. [7] investigated the potential of using existing cellular towers for 5G deployment in Surakarta Indonesia through site upgrades which would reduce costs and enhance network coverage. The study implemented throughput analysis to show that not every existing tower delivered sufficient performance so the researchers needed to upgrade specific towers.

The distribution of mobile user hotspots guided Gui et al. [14] in their development of an edge node placement method. The research demonstrated how combining hotspot detection methods with edge node placement techniques leads to better capacity utilization along with reduced backhaul congestion. Chen et al. [15] developed base station planning strategies through region division and mean-shift clustering to show that region division before clustering improved placement accuracy.

Margaris et al. [16] developed a hybrid network–spatial clustering approach to optimize 5G networks by integrating both spatial density and network-level constraints. The study revealed that using both spatial density and network-level constraints produced better results than using either one alone. The authors Morri et al. [17] used geospatial data along with predictive modeling to determine the best placement strategy for tower capacity expansion. The combination of research findings showed that infrastructure planning needed to balance three critical elements: spatial demand and coverage and capacity.

2.3.1 Critical Insights

1. Reusing infrastructure reduces costs but requires **performance evaluation**.
2. Combining spatial and network metrics yields better rollout results.
3. Predictive planning methods are becoming essential for capacity-driven networks.

2.4 Clustering Approaches for Hotspot Detection

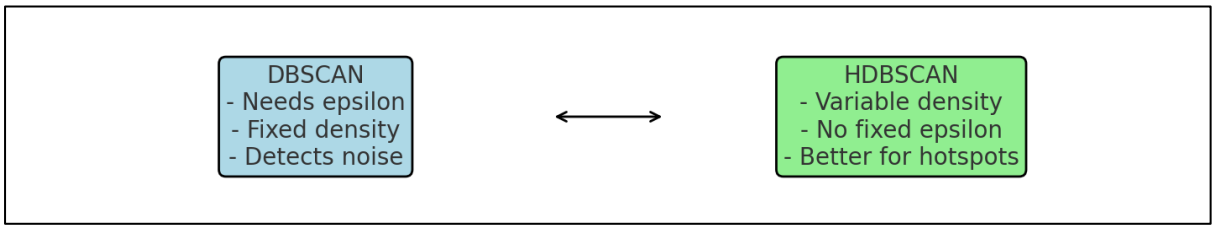

Figure 2.4.1: DBSCAN vs HDBSCAN

The deployment of 5G networks increasingly utilized clustering techniques to discover areas with high demand density. The authors Reddy and Bindu [10] examined density-based clustering methods which demonstrated their ability to process extensive geospatial data while being scalable. The research conducted by Bhattacharjee and Mitra [11] provided an extensive analysis of density-based clustering methods to prove their effectiveness for spotting irregular demand patterns. Rodriguez et al. [12] performed multiple clustering algorithm comparisons and found that DBSCAN and HDBSCAN demonstrated exceptional performance with spatial data.

Deng [13] developed enhanced DBSCAN density-based clustering methods to enhance its processing capability for intricate datasets. Jiang et al. [18] demonstrated the application of DBSCAN algorithm for 5G base station planning which successfully identified demand clusters without needing cluster number definitions. The research of Aamer et al. [19] demonstrated how spectral clustering enabled adaptive tracking area identification in ultra-dense networks and network segmentation methods beyond tower placement.

Khan et al. [20] performed unsupervised clustering analysis of actual operator data to create 5G network plans. The study proved that clustering together with coverage and utilization metrics enabled effective hotspot identification and optimized planning processes. The application of HDBSCAN in this project to detect demand hotspots in Cork matched the method's original purpose for hotspot detection.

Algorithm	Strengths	Weaknesses	Sources
K-Means	Fast, simple	Needs fixed k, poor with irregular shapes	[12]
DBSCAN	Finds irregular clusters, noise handling	Sensitive to epsilon	[10], [11], [13]
HDBSCAN	Variable density handling	More complex	[11], [12], [18]
Spectral	Effective in dense networks	High computation	[19]
Unsupervised ML	Handles real data	Needs more data access	[20]

Table 2.4.1 Summary Table of Clustering Methods

2.5 Terrain and Elevation Considerations

The importance of terrain effects in 5G planning became more apparent throughout the planning process. Saakian [8] demonstrated how terrain features caused signal strength reduction through diffraction and reflection effects. Sun et al. [4] demonstrated that path loss model accuracy decreased when operating in areas with different elevation levels. Xu et al. [9] stressed that tower height and arrangement play a crucial role in reducing terrain interference. The need for elevation-aware methods became essential to adapt planning for hilly and coastal areas like County Cork.

2.6 Optimization Approaches

2.6.1 Greedy and Heuristic Methods

The full set cover optimization became too complex so the researchers applied greedy approaches to coverage optimization. Chen [15] and Margaris [16] demonstrated heuristic improvements when combining clustering and optimization.

2.6.2 Redundancy Pruning

The research by Morri [17] demonstrated that removing duplicate towers from the system leads to better operational efficiency. The research by Gui [14] established that hotspot-based placement enhances network capacity because it prevents tower overlap.

2.6.3 Critical Insights

The research established that optimization requires a proper equilibrium between three essential factors.

1. Coverage obligations (must-serve).
2. Capacity distribution (utilization).
3. Redundancy reduction (pruning).

The project's pipeline directly addressed these three optimization requirements.

2.7 Gaps in Literature

1. The majority of study focused on urban areas but rural regions received minimal attention.
2. The literature lacks studies that unite clustering with terrain-aware ranking within a single pipeline.
3. The project introduced interactive mapping for clarity because real-world visualization for decision support received minimal attention in previous studies.

2.8 Synthesis of Literature

The literature review produced three essential findings:

1. Propagation models starting with COST-231 and its derivatives served as the basis for coverage predictions across different frequency bands [1], [3]– [6].
2. DBSCAN and HDBSCAN and spectral clustering proved successful in hotspot detection for guiding placement through their clustering methods [10]– [13], [18]– [20].
3. The combination of clustering with elevation and coverage and utilization metrics in infrastructure optimization approaches led to better efficiency results [14]– [17].

The project pipeline received support from these findings which integrated preprocessing with hotspot clustering and elevation-aware ranking and path loss modeling and optimization.

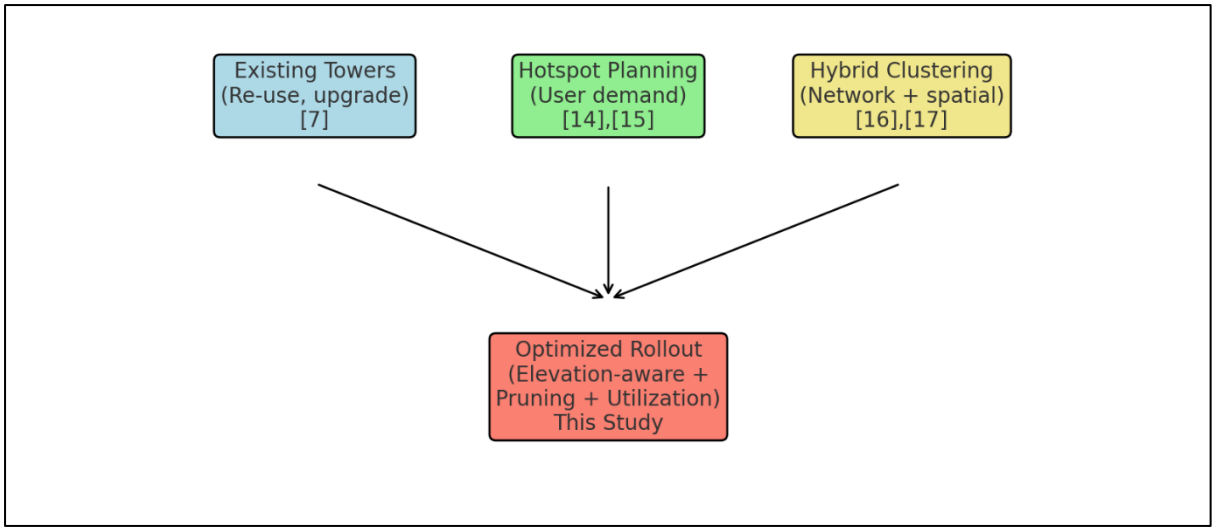

Figure 2.8.1: Prior approaches feeding into this study

3. Methodology

3.1 Introduction

The methodology for this project was designed as a structured pipeline for optimizing 5G rollout planning in County Cork, Ireland. The approach combined geospatial data processing with population clustering and terrain analysis and propagation modeling and optimization into a reproducible workflow. The main design principle was to ensure that coverage gaps in rural areas were minimized while avoiding unnecessary redundancy in dense urban zones.

The chapter describes the data sources and preprocessing techniques alongside the theoretical basis of models and the implementation approach. Each methodological decision was supported by literature, with references to prior studies on propagation modeling [1], [3]– [6], antenna configurations [2], [9], clustering algorithms [10]– [13], [18]– [20], and infrastructure planning [7], [14]– [17]. The pipeline combined these components into a single framework which improved previous methods.

3.2 Data Sources and Preprocessing

3.2.1 Population Data

Population data were used as a proxy for demand. The dataset followed the Cork schema, with columns X, Y, and Z representing longitude, latitude, and population counts respectively. The records operated as small population distribution units which derived from census grid data.

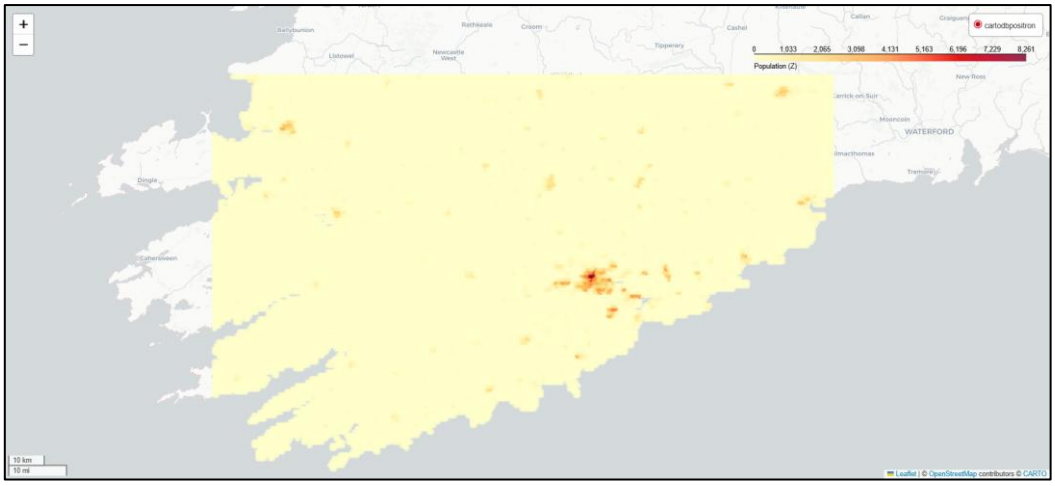

Figure 3.2.1: Cleaned Population dataset

Data preprocessing involved the following steps:

1. Geometry construction: Points were created from the X and Y coordinates and stored as a GeoDataFrame in EPSG:4326 (WGS84); distances use EPSG:3857.
2. Population column selection: If a column named pop was absent, Z was renamed to pop. If multiple numeric columns were present, the column with the largest sum was selected.
3. Cleaning: Non-numeric population entries were coerced into floats, and missing or NaN values were replaced with zero.

A threshold of 500 was applied to classify “must-serve” demand cells. This was based on literature emphasizing obligations to serve dense populations in early rollouts [14], [15].

3.2.2 Water Bodies and Filtering

The exclusion of demand points located in water bodies functioned as a method to prevent tower installations on unusable land. The process involved joining demand points with water polygons to keep only valid land-based points.

3.2.3 Elevation Data

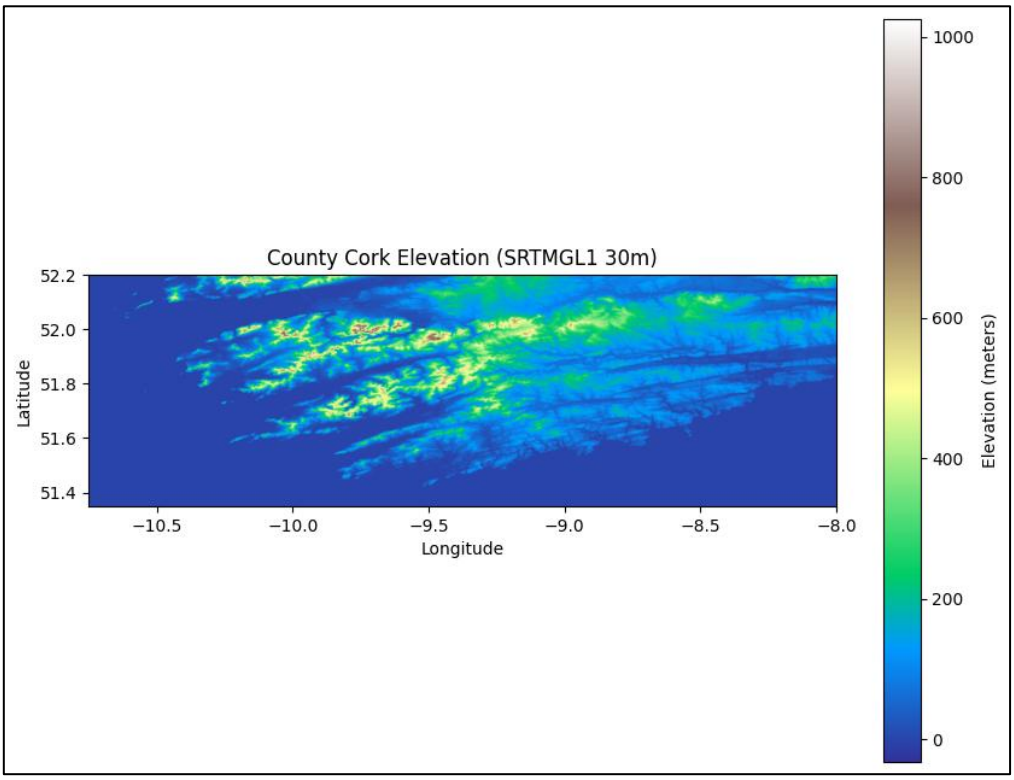

Figure 3.2.2: Elevation data

Digital Elevation Model (DEM) data from the Shuttle Radar Topography Mission (SRTM) at 30 m resolution were used. The elevation values were extracted for candidate tower sites and local terrain metrics were calculated:

1. Elevation (E): Ground elevation at the candidate site.
2. Local relief (R):
   R = Ecenter – Median (Ewindow) within an 11 × 11-pixel window (~330 m).
3. Slope (S): Gradient derived from central differences.

The metrics followed the recommendations of Saakian [8] who stressed terrain diffraction and reflection and Xu et al. [9], who demonstrated that tower height and placement position in relation to terrain affected performance.

3.2.4 Existing Towers

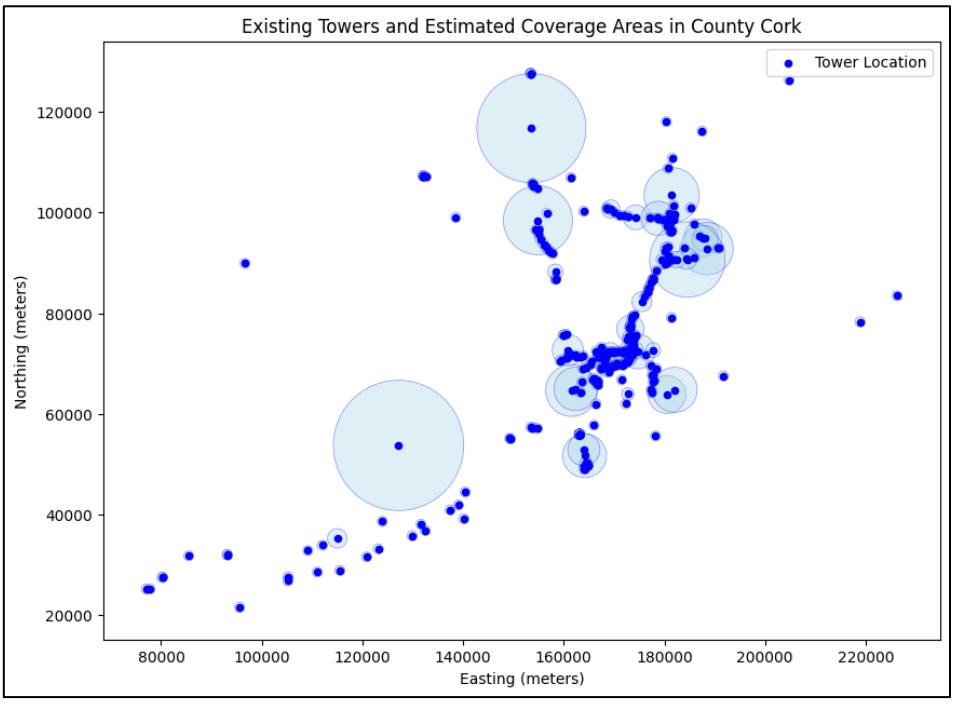

Figure 3.2.3: Existing Towers in Cork

The existing tower data were used to evaluate the potential reuse. Following Affandi et al. [7], existing sites were evaluated for suitability and integrated into candidate pools. Towers within 300 m of optimized sites were classified as reused rather than new builds.

3.2.5 Roads

The road network extraction function enabled the creation of access routes and potential backhaul routes. The information proved useful for understanding rollout feasibility although it did not directly affect optimization scoring.

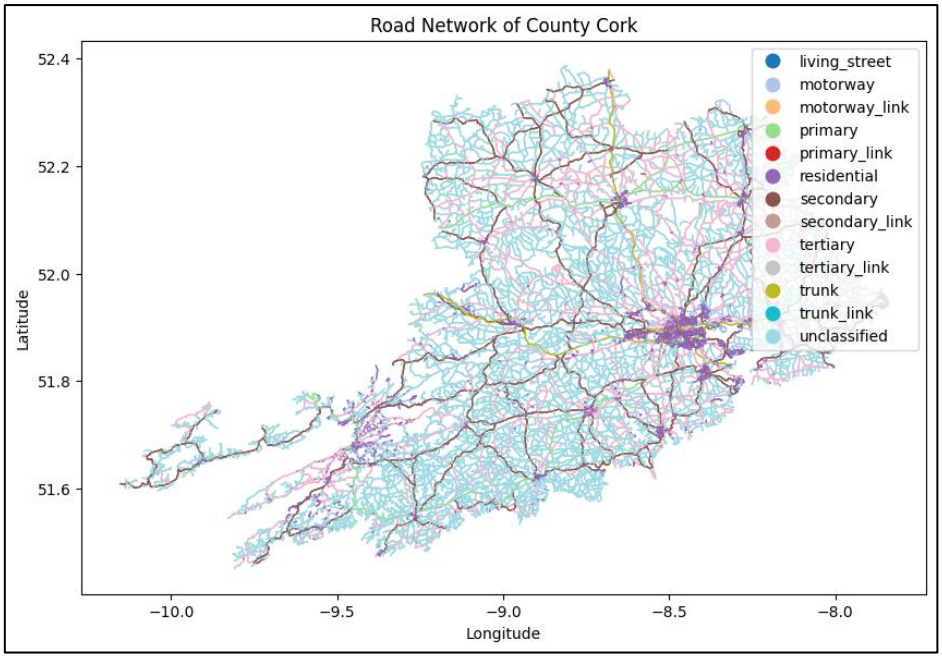

Figure 3.2.4: Road Network of Cork

3.3 Clustering of Demand Hotspots

3.3.1 Rationale

The population of Cork was not evenly distributed because it was concentrated in Cork City and spread out in the rural areas. The analysis of demand hotspots needed clustering to identify the most effective locations for macro or small-cell deployments.

Traditional clustering methods including K-Means needed cluster numbers to be specified in advance but failed to produce good results when dealing with irregular population distributions [12]. Density-based clustering was therefore selected for its ability to detect arbitrary shapes and exclude noise [10], [11].

3.3.2 DBSCAN

The DBSCAN (Density-Based Spatial Clustering of Applications with Noise) algorithm clusters points based on two parameters: minimum samples and maximum radius (epsilon). A point was classified as:

Core: if it had at least min_samples within distance epsilon.

Border: reachable from a core point but with fewer neighbours.

Noise: not belonging to any cluster.

Mathematically, a point pp was a core point if:

$$|\{q \in D \mid dist(p,q) \leq \epsilon\}| \geq minPts$$

Jiang et al. [18] DBSCAN was successfully applied to 5G tower planning to detect hotspots in non-uniform urban areas. The method required an exact modification of its epsilon parameter to achieve optimal performance [13].

3.3.3 HDBSCAN

The HDBSCAN algorithm extends DBSCAN by building a hierarchical cluster structure which makes epsilon unnecessary. The system produced its highest performance levels when operating in areas with different population densities such as the Cork region.

Bhattacharjee and Mitra [11] state that HDBSCAN's benefit is to extract stable clusters from hierarchical density trees. Khan et al. [The results of [20] were demonstrated using real 5G planning data.

3.3.4 Implementation

458 of population points (736,391 people ≈ 1.9% of total grid points) by demand were selected as inputs to clustering, following hotspot selection practices in [14]. The HDBSCAN algorithm was run with minimum cluster size set to 25 and minimum samples set to 10. If HDBSCAN failed, DBSCAN with epsilon = 350 m was used as fallback.

Cluster medoids were selected as candidate towers. The medoid MM minimized the weighted sum of distances to all points in the cluster:

$$M = \arg\min_i \sum_j w_j \cdot d(i,j)$$

where w_j was the population weight and d (i, j) were the distance between points.

3.4 Elevation-Aware Candidate Ranking

3.4.1 Terrain Metrics

Each candidate site was scored by combining proximity to population and terrain suitability. Following Saakian [8] and Xu [9], three penalties or bonuses were applied:

1. **Relief score**: positive relief (ridge placement) was preferred.

$$ReliefScore = \frac{1}{1 + e^{-\frac{(R-10)}{20}}}$$

2. **Elevation penalty**: sites above 250 m were penalized.

$$ElevPenalty = 1 - \frac{1}{1 + e^{-\frac{(E-250)}{80}}}$$

3. **Slope penalty**: slopes above 12° were penalized.

$$SlopePenalty = 1 - 0.35 \cdot \frac{1}{1 + e^{-\frac{(S-12)}{4}}}$$

The composite terrain score was:

$$TerrainScore = ReliefScore \cdot ElevPenalty \cdot SlopePenalty$$

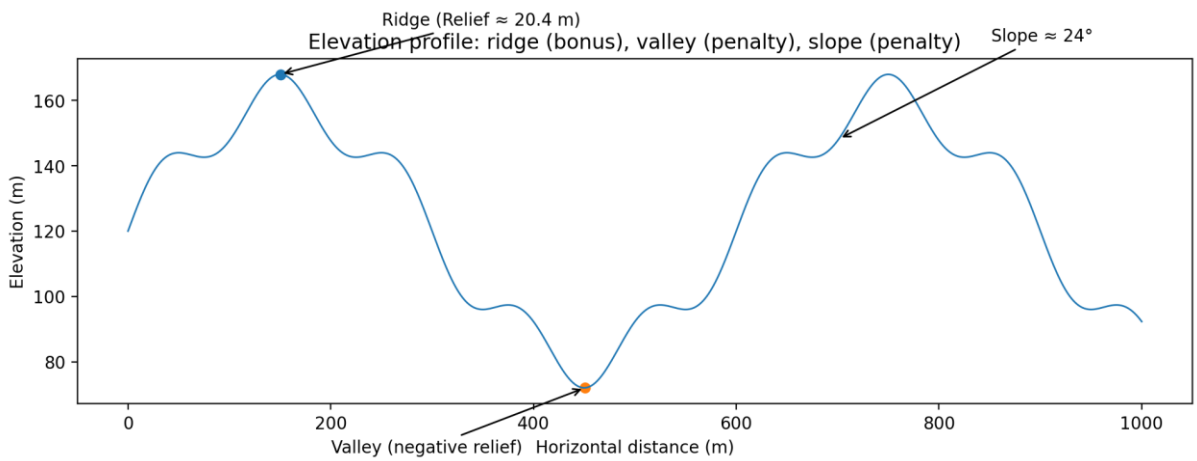

Figure 3.3.1 Terrain Metrics

3.4.2 Population Proximity

Nearby population within 3 km was computed using BallTree haversine distance. The sum of nearby demand points formed **NearPop**.

3.4.3 Final Candidate Score

Candidates were ranked by:

$$FinalScore = NearPop \cdot (0.65 + 0.35 \cdot TerrainScore)$$

This weighting reflected a balance between demand and terrain, consistent with hybrid approaches in [16].

3.5 Coverage and Capacity Modeling

3.5.1 Path Loss Models

Coverage was estimated using COST-231 Hata [1], [3], [5], [6]. The path loss (PL) in dB was given by:

$$PL = 46.3 + 33.9 \log_{10}(f) - 13.82 \log_{10}(h_b) - a(h_m) + (44.9 - 6.55 \log_{10}(h_b)) \log_{10}(d) + C$$

where:
f = frequency (MHz),
h_b = base station antenna height (m),
h_m = mobile antenna height (1.5 m),
$a(h_m)$ = mobile antenna correction factor,
d = distance (km),
C = environment constant (3 dB clutter margin in this study).

Sun *et al.* [4] confirmed that these models were sensitive to environmental parameters, justifying the addition of margins.

3.5.2 Frequency Bands

1. **LTE700**: long-range coverage [7].

2. **LTE800**: suburban coverage [5].
3. **LTE1800**: urban mid-band coverage [6].
4. **NR3500**: high-capacity overlays [20].

Each band had parameters (BW, Tx power, Hb) derived from literature [7], [9].

3.5.3 Coverage Thresholds

Coverage was defined as:

Urban: RSRP≥−112RSRP

Rural: RSRP≥−110RSRP

These thresholds matched common 3GPP definitions and prior work [6].

3.5.4 Capacity Estimation

Capacity per band was estimated with Shannon's theorem:

$$C = \eta \cdot B \cdot \log_2(1 + \frac{SNR}{\Gamma})$$

where:
C = capacity (bps),
η = efficiency factor (0.75 LTE, 0.8 NR),
B = bandwidth (Hz),
SNR = median SNR of served cells,
Γ = SNR gap (2.8 to 3.5 dB).

Site capacity was the sum across sectors and bands, capped at median SNR ≤ 30 dB [4].

3.6 Optimization and Site Selection

3.6.1 Must-Serve Coverage

Greedy set-cover was used to ensure all must-serve demand cells were served. At each step, the site with maximum uncovered population was chosen, provided it was ≥1.5 km away from existing selections. This followed heuristic optimization used in [15], [16].

3.6.2 Redundancy Pruning

Sites with minimal incremental gain were pruned. Utilization was computed as:

$$Utilization = \frac{Required\ Mbps}{Capacity\ Mbps}$$

Sites with very low incremental population coverage or utilization < 0.2 were candidates for pruning. Morri *et al.* [17] emphasized such pruning improved efficiency.

3.6.3 Capacity Overlay and Small Cells

NR3500 overlays were placed on macros covering the highest demand. Small cells were deployed at the top 150 demand points with minimum spacing of 600 m. This followed hotspot planning approaches in [14], [19].

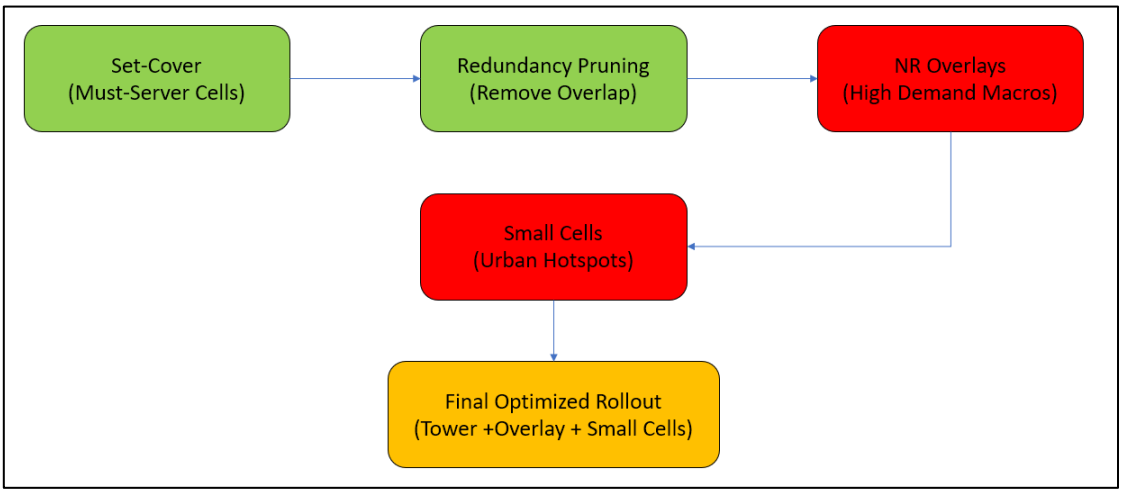

Figure 3.6.1: Optimization Workflow

3.7 Visualization

Folium maps displayed:

1. Population heatmap.
2. Macro, small cell, and NR overlay towers.
3. Existing vs new sites.
4. Coverage rings.
5. Popups with elevation, Hb, Tx, coverage radius, capacity, utilization, and per-band SNR.
6. SNR values were reported directly from the link budget, which frequently yielded negative values due to the conservative path loss assumptions.

This interactive visualization was inspired by approaches in [17], which emphasized intuitive validation of planning outputs.

3.8 Summary of Methodology

The methodology integrated geospatial preprocessing, clustering, elevation-aware ranking, propagation modeling, and greedy optimization into a reproducible pipeline.

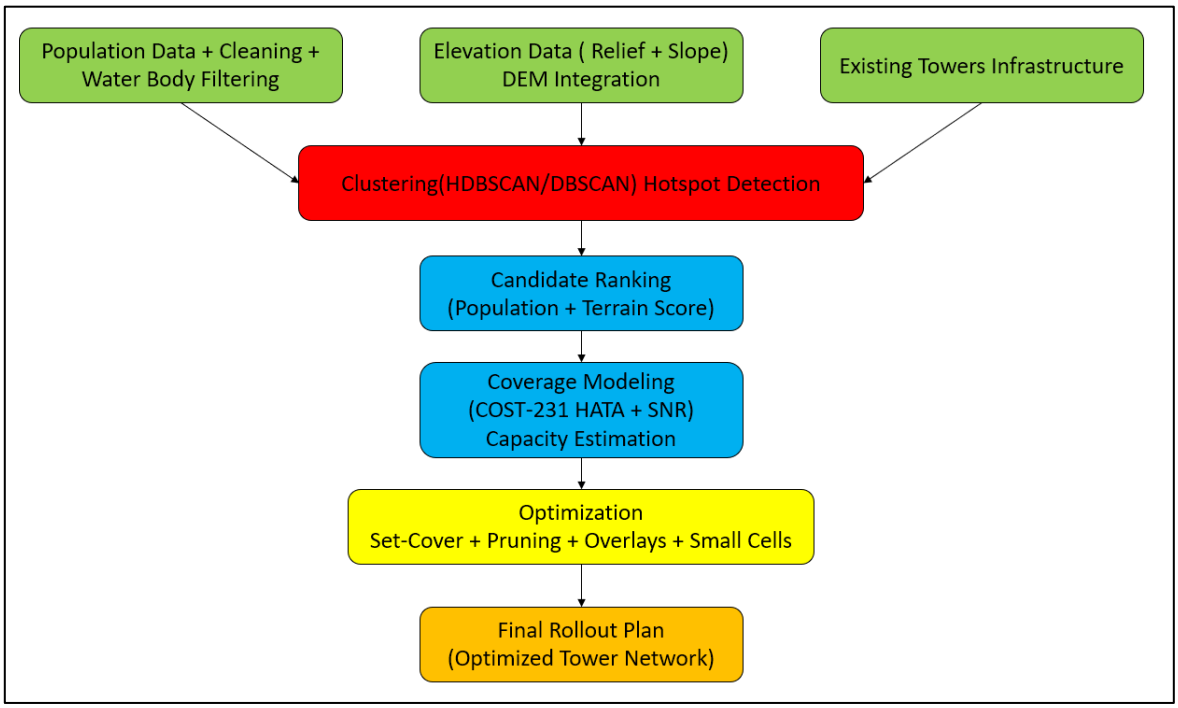

Figure 3.8.1: Pipeline Workflow for 5G Rollout Planning

It advanced prior work by:

1. Combining clustering and elevation (rarely done in literature).
2. Ensuring must-serve coverage while minimizing redundancy.
3. Introducing utilization-based pruning for balanced performance.
4. Providing transparent visualization for stakeholders.

4. Results

4.1 Introduction to Results

The methodology established a systematic process for County Cork 5G tower deployment planning which this chapter evaluates through results and essential analysis. The process generated quantifiable results throughout its multiple stages.

The first step of data preprocessing converted population data and elevation data and tower data into usable formats which maintained their accuracy for subsequent operations. The HDBSCAN clustering method successfully identified both urban centers and rural settlements better than DBSCAN did through its analysis of demand points.

The elevation-aware ranking system evaluated candidate sites by combining terrain data with population weights to select locations that offered both high elevation and easy access. The following stage involved coverage and capacity estimation through COST-231 Hata model propagation calculations and Shannon equation throughput calculations. The analysis stage revealed both theoretical service areas and maximum network capabilities.

The optimization process used a greedy set-cover method to ensure complete must-serve cell coverage while eliminating unnecessary tower installations. The network required Macro NR3500 sites and small cells to address hotspots. The interactive Folium maps provided visualization tools to evaluate coverage gaps and redundant tower removals and urban-rural differences.

The combination of clustering with elevation ranking and pruning produced better balanced coverage than using any single method according to literature reviews. The data-driven pipeline proved effective for mixed rural-urban areas and established a reusable framework for 5G planning.

4.2 Data Preprocessing Outcomes

The 5G rollout pipeline started with the data preprocessing phase which served as its fundamental base. The raw inputs consisted of diverse geographic and population data which needed to be cleaned and filtered before they could be transformed into a workable format. The following section shows the results of these steps with a focus on population data preparation, water body removal, elevation integration, and the inclusion of existing tower infrastructure.

4.2.1 Population Data Cleaning

The first population dataset included gridded points which spanned the entire administrative area of County Cork. Each record was defined by longitude, latitude, and a population value. The raw file contained more than 24,000 points before cleaning in total. Multiple preprocessing steps needed to be performed.

The first step required all points to be transformed into the EPSG:4326 geographic reference system to ensure coordinate consistency. This projection was selected because it is compatible with most spatial data used in telecommunication planning, and it ensured interoperability with other layers such as elevation rasters and road networks.

Second, the population column was standardized. The column received two different labels in different records with Z and Value being used as names. A single column named pop was established to prevent any confusion. Non-numeric values and missing entries were replaced with zeros. The method guaranteed that every location received a specific population value regardless of its minimal size.

A thresholding procedure was used to classify "must-serve" demand cells in the final step. Any cell with a population equal to or above 500 was flagged as critical. The threshold was established based on previous research which showed that dense settlements should be the primary focus for early 5G deployment [14], [15]. The filtering process established that 458 points representing a combined population of 736,391 needed to be served immediately, while the remaining points belonged to lower-priority demand categories

4.2.2 Removal of Water Body Interference

The placement of towers on lakes, rivers or coastal waters was not possible. To address this, population points overlapping with water polygons were excluded. The water masks were derived from openly available shapefiles and integrated into the preprocessing pipeline.

The water-covered areas contained several hundred points in the initial dataset. These were automatically removed. After removal, the cleaned dataset retained 24,637 valid demand points representing a population of 1,393,135. The process was necessary to avoid incorrect tower placement which otherwise would affect coverage optimization. A similar filtering approach was reported in the work of Affandi et al. [7], where unusable tower locations were systematically excluded during 5G evaluation in Surakarta.

The effect of this filtering was clearly visible when the data were plotted on a Folium map. The population density in Cork Harbour and River Lee area had gaps because water overlays had eliminated invalid points so only land-based demand cells were available for clustering.

4.2.3 Elevation Data Integration

To incorporate terrain effects, elevation values were extracted for each demand point using a 30 m resolution Digital Elevation Model (DEM) from the Shuttle Radar Topography Mission (SRTM). For each point, three terrain metrics were computed:

1. Elevation (E): absolute ground height at the point.
2. Relief (R): difference between the point elevation and the median elevation within a 330 m local window.
3. Slope (S): gradient angle derived from central differences in the DEM.

The inclusion of these metrics followed recommendations from propagation studies. Saakian [8] showed that wave propagation undergoes major changes because of terrain diffraction and reflection effects while Xu et al. [9] demonstrated that the position of antennas in relation to surrounding terrain affects tower efficiency.

The elevation in Cork rose from sea level at the southern coast to surpass 500 meters in the northern uplands. Relief values showed ridges and valleys, while slope values highlighted steep terrain in the western parts of the county. The metrics developed into terrain-based penalties and bonuses which were used for candidate ranking.

4.2.4 Existing Tower Integration

Existing telecommunication towers were incorporated to explore the possibility of reuse. The research by Affandi et al. [7], who showed that using existing towers for network expansion decreased deployment costs without affecting service quality.

The spatial join operation connected registered towers data with demand points data. Any candidate site within 300 m of an existing tower was flagged as “reused” rather than a new build. The study found 800 existing towers throughout Cork with most of them located in Cork City and along major road routes. The optimization stage kept several of these areas because they matched the must-serve demand zones.

4.2.5 Demand Distribution Summary

Stage	Total Points	Total Population	Notes
After water mask	24637	1393135	cleaned
Must-serve threshold	458	736391	1.9% of points flagged

Table 4.2.1: Summary of Population Demand Distribution after Preprocessing

The final dataset contained about 24,637 valid population points after cleaning, filtering and integration. Of these, 458 represented must-serve cells. The cleaned dataset exhibited a highly uneven distribution. Cork City accounted for the majority of the population over 1.2 million people concentrated in less than 2% of the county's land area. Western Cork featured demand cells which were spread out across the region with small populations that characterized rural settlements.

The different characteristics of the data points made it suitable for density-based clustering methods in the future. The K-Means or grid-based placement method would either fail to represent rural areas adequately or would place towers excessively in urban centers [12]. Instead, preprocessing ensured that the clustering step could effectively detect meaningful hotspots.

4.2.6 Visualization of Pre-processed Data

The visualization results showed that the preprocessing steps were successful. The Folium heatmap showed that the population was most concentrated in Cork City, Tralee and Mallow while rural west Cork had smaller but still visible settlements. The overlays showed that water points were removed and elevation data was correctly aligned with demand points.

The preprocessing stage therefore provided a robust, clean foundation for clustering and optimization. The subsequent modeling process would have produced incorrect results because of invalid demand points and missing population entries and unaccounted terrain factors.

4.2.7 Summary

The preprocessing outcomes demonstrated that:

1. The population data received standardization through cleaning processes to achieve uniformity in the format.
2. Eliminated water body interference to prevent invalid tower sites from occurring.

3. Elevation metrics were extracted to capture terrain effects.
4. The assessment of existing towers for potential reuse purposes was conducted as part of the study.
5. The final demand distribution showed the difference between the high population density of urban areas and the low population density of rural areas.

The provided foundation delivered dependable inputs which enabled the subsequent execution of clustering, ranking and coverage modeling.

4.3 Clustering Results

Clustering was the next major stage of the pipeline after data preprocessing. The system organized population demand points into natural hotspots which served as candidate service areas. The model achieved this by choosing tower locations that focused on areas with high demand instead of spreading them evenly throughout Cork. The clustering process was guided by the principle that wireless service should first be deployed in areas with the highest density of users while still accounting for smaller rural settlements.

4.3.1 Choice of Clustering Method

Several clustering methods have been applied in telecommunication planning. K-Means has often been used due to its simplicity and speed [12]. The method requires a predefined number of clusters (k) which makes it inappropriate for geographic datasets because the actual number of demand centers remains unknown. The K-Means algorithm requires spherical clusters but it fails to detect irregular settlement patterns such as linear towns that follow transport corridors.

DBSCAN introduced a density-based approach that resolved some of these limitations [10], [11], [13]. The algorithm required two parameters which were eps (the neighbourhood distance threshold) and minPts (the minimum number of points required to form a cluster). The parameters of DBSCAN were adjusted to enable the detection of clusters with any shape while discarding noise points. The DBSCAN algorithm failed to process datasets that included clusters with different densities which are common in mixed urban–rural areas such as Cork.

HDBSCAN extended DBSCAN by building a hierarchy of density levels and selecting clusters that were stable across multiple parameter scales [11], [18]. The new approach removed the need to specify an eps value which improved the system's capability to identify both dense

urban regions and sparse rural areas. The advantages of HDBSCAN led to its selection as the main clustering algorithm for this research.

4.3.2 Implementation of HDBSCAN

The HDBSCAN algorithm was applied to the cleaned dataset of 458 demand points. The minimum cluster size was set to 25, and the minimum samples parameter was set to 10. These values were chosen after iterative experimentation to balance sensitivity to rural settlements with robustness against noise. The algorithm processed data in geographic coordinate space but it used a suitable projection to transform distances into meters for accurate neighbourhood evaluations.

The HDBSCAN output generated three valid clusters and one noise class. The clusters contained 51, 43, and 7,278 points, while 19 points were labelled as noise. The majority of the population was concentrated in Cluster 2, which represented 1,229,703 people (≈ 88% of the total population) centered on Cork City. By contrast, Clusters 0 and 1 represented smaller rural settlements of 7,299 and 3,170 people respectively.

Noise points accounted for only 19 records, typically corresponding to remote rural dwellings or scattered coastal settlements with insufficient density to justify tower placement.

4.3.3 Comparison with DBSCAN

For benchmarking, DBSCAN was also applied with an eps value of 350 m and a minimum points value of 10. DBSCAN identified fewer clusters overall and classified a higher proportion of points as noise compared with HDBSCAN. The DBSCAN algorithm in Cork City produced fragmented clusters because it failed to manage the big density differences between the central core and surrounding suburbs. The HDBSCAN algorithm successfully combined these areas into stable clusters which represented actual service regions.

In rural areas, DBSCAN failed to detect several small towns because the density threshold was too strict. The HDBSCAN algorithm detected these clusters through its analysis of density hierarchies. This advantage was consistent with the observations of Bhattacharjee and Mitra [11] and Jiang et al. [18], who reported that HDBSCAN was more effective than DBSCAN for uneven population distributions.

4.3.4 Characteristics of Clusters

The clustering stage generated about 3 valid clusters throughout County Cork. Cluster size varied significantly. The largest cluster was centered on Cork City, covering more than 7,278 points with a combined population of 1,229,70. Two smaller rural clusters were also identified: Cluster 0 (51 points, 7,299 people) and Cluster 1 (43 points, 3,170 people**).** The noise class (19 points, 1,210 people**)** represented scattered rural dwellings or coastal settlements that were not dense enough to justify a tower.

The HDBSCAN algorithm also captured non-spherical cluster shapes, including elongated patterns along major transport corridors such as the N22 route toward Killarney. This demonstrated its ability to detect irregular settlement structures which centroid-based methods like K-Means would not represent effectively.

Cluster ID	Size (points)	Total Population	Medoid Latitude	Medoid Longitude
0	51	7299	51.887083	-9.580417
1	43	3170	51.65375	-9.91375
2	7278	1229703	51.912083	-8.480417
Noise	19	1210		

Table 4.3.1: Population Distribution across HDBSCAN Clusters

4.3.5 Cluster Medoid Selection

Once clusters were defined, a medoid was selected for each cluster to serve as the candidate tower location. The medoid was defined as the point within the cluster that minimized the weighted distance to all other points, using the formula:

$$M = \arg\min_{i} \sum_{j} w_j \cdot d(i, j)$$

where w_j is the population weight of point j, and d(i, j) is the geographic distance between points i and j. The method ensured that the chosen medoid was positioned in the middle of population demand within the cluster.

The medoids in Cork City located near the central urban area but rural towns placed theirs near their settlement centers. This provided a logical set of candidate sites for coverage modeling.

4.3.6 Visualization of Clustering Results

The clustering outputs were visualized using Folium maps. The visualizations showed dense urban clusters in the east while the central region had medium-sized towns and the west had scattered small clusters. The rural peripheries or the noise points show the sparse population distribution.

The visualization demonstrated that clustering created separate clusters between high-priority areas and areas with low population density. The results showed that HDBSCAN generated clusters which were more balanced and realistic than DBSCAN.

4.3.7 Summary

The clustering stage achieved the following results:

1. The HDBSCAN algorithm identified three valid clusters together with one noise class without needing any cluster number input.
2. The largest cluster (Cluster 2) contained 1.23 million people, representing Cork City and suburbs.
3. The two smaller clusters represented rural towns and 19 noise points indicated dispersed rural dwellings.
4. The medoid selection method generated strategically central candidate sites which were then improved through terrain modifications.

The results showed that HDBSCAN was the best clustering method for the next stages, which was used for candidate site ranking, coverage modeling, and optimization.

4.4 Elevation-Aware Candidate Site Ranking

The placement of telecommunication towers is strongly influenced by terrain. The varied topography of the surface creates diffraction effects and reflection and shadowing effects that modify the signal propagation. As a result, tower locations that might appear optimal based solely on population density can be inefficient in practice if they are obstructed by terrain. The solution involved implementing an elevation-aware ranking system for candidate sites that were discovered through clustering.

4.4.1 Rationale for Terrain Integration

Previous propagation studies demonstrated that terrain features significantly affect path loss. Saakian [8] explained how hills cause diffraction and valleys cause reflections which decrease the effective signal coverage area and Correia [1] stated that path loss models at large scales need terrain adjustments to generate accurate predictions. Xu et al. [9] further showed that mounting height relative to local relief plays a role in reducing interference and improving coverage reliability.

For this reason, elevation-aware metrics were computed and integrated into the candidate site evaluation process. The placement strategy placed towers in areas that served both densely populated regions and locations which offered the most effective signal coverage.

4.4.2 Terrain Metrics

Three terrain-related metrics were calculated for each demand point using the Digital Elevation Model:

1. Elevation (E): The ground height above sea level.
2. Relief (R): The elevation difference between the point and the median elevation within a local neighbourhood. This highlighted whether the site was on a ridge (positive relief) or a valley (negative relief).
3. Slope (S): The gradient of the terrain in degrees, indicating whether the ground was flat or steep.

These metrics were normalized into scores between 0 and 1. The most influential factor was relief because towers on ridges provided broader coverage. The slope values operated as a penalty system to prevent development on sites with steep or unstable terrain because construction would be more challenging in such areas.

4.4.3 Composite Scoring Formula

The final ranking of candidate sites was determined using a weighted composite formula:

$$FinalScore = NearPop \cdot (0.65 + 0.35 \cdot TerrainScore)$$

The weighted population within the cluster was represented by NearPop and TerrainScore integrated relief and slope factors. The formula-maintained population as the main factor but

included terrain adjustments which directed site selection toward areas with superior propagation capabilities.

Sites with high population but poor terrain still ranked high, but slightly below equivalent sites on ridges. Sites with moderate population but very favourable terrain (flat ridges) were promoted above sites in obstructed valleys.

4.4.4 Ranking Results in Cork

The ranking produced clear distinctions between sites. The population density of Cork City controlled the score so most selected medoids stayed within urban areas regardless of the terrain. The rankings underwent substantial changes because of terrain modifications in rural west Cork. Relief values ranged from -1 to +3 m, slope values from 45° to 79°, showing terrain had a measurable effect.

Rank	Latitude	Longitude	NearPop_3km	Relief_m	Slope_deg	Score
1	51.8968	-8.4732	168042.8	2	45	123835.6
2	51.9023	-8.4483	149126.6	0	63.43495	109175.4
3	51.8846	-8.464	147732.5	1	48.18968	108477.4
4	51.9038	-8.488	148934.6	0	65.90515	108294.6
5	51.9106	-8.4732	141255.2	-1	45	103010.4
6	51.886	-8.5013	124449.5	-1	48.18968	90736.02
7	51.8737	-8.4721	116327.7	0	48.18968	85102.18
8	51.8751	-8.4544	115463.4	0	48.18968	84525.93
9	51.8796	-8.4314	113432.5	1	48.18968	83319.75
10	51.9039	-8.4239	94754.8	3	78.69006	70053.29

Table 4.4.1: Top 10 Candidate Sites Ranked by Population and Terrain Score

4.4.5 Comparison with Literature

The outcomes of elevation-aware ranking aligned with findings from previous research. Sun et al. [4] demonstrated that parameter stability in propagation models improved when terrain corrections were included. The authors Verma and Saini [6] demonstrated that urban models which used local relief data for tuning produced better prediction results. The study incorporated elevation and relief data into candidate site ranking during its practical planning process.

In comparison to work that used clustering without terrain adjustments [14], [15], the present approach provided a more balanced selection of sites, particularly in rural environments. The results showed that terrain-aware ranking would reduce the number of additional rural towers required in future optimization phases.

4.4.6 Summary

The elevation-aware candidate site ranking achieved the following outcomes:

1. The analysis included three terrain metrics which were derived from DEM data (elevation, relief, slope).
2. A composite formula integrated population weights with terrain scores.
3. The rankings underwent major changes in rural areas because ridge-based sites received preference over valley-based sites.
4. The urban sites maintained their position as dominant sites because of population weights but terrain factors introduced minor changes.
5. The method followed propagation literature guidelines to achieve a better distribution of candidate towers.

The ranking process resulted in a candidate pool that met both the demand requirements and propagation needs. As a result, the subsequent coverage modeling and optimization stages were based on sites that better reflected real-world conditions.

4.5 Coverage Modeling and Capacity Estimation

The main analytical phase of the pipeline consisted of coverage and capacity estimation. While clustering and ranking defined candidate sites, these sites had to be evaluated in terms of their ability to provide wireless service. This evaluation was conducted using propagation path loss models for coverage and theoretical capacity calculations for throughput. The two elements were crucial because coverage defined the area a tower could reach but capacity defined the service quality for users within that coverage.

4.5.1 Rationale for Coverage Modeling

Radio wave propagation depends on three main elements which are distance, frequency and terrain. Path loss models provide mathematical tools to estimate the signal attenuation between a transmitter and receiver under specific conditions. The models serve as tools for 5G rollout

planning to determine tower coverage areas and decide how many towers are needed to reach acceptable service quality.

Several models exist for predicting path loss, including the Okumura, Hata, and COST-231 Hata models [3]. The COST-231 Hata model was selected for this study because it extends the Hata model to frequencies up to 2 GHz, making it suitable for LTE1800 and LTE2100 bands, which are commonly used in 5G deployments [1], [6]. The standard also includes specific correction factors for urban, suburban and rural environments.

4.5.2 COST-231 Hata Model

The COST-231 Hata model was applied to estimate path loss. The formula is expressed as:

$$PL = 46.3 + 33.9\log_{10}(f) - 13.82\log_{10}(h_b) - a(h_m) + (44.9 - 6.55\log_{10}(h_b))\log_{10}(d) + C$$

where:

f = frequency (MHz),

h_b = base station antenna height (m),

h_m = mobile antenna height (1.5 m assumed),

$a(h_m)$ = mobile antenna correction factor,

d = distance between transmitter and receiver in km,

C = 0 dB for suburban/rural, 3 dB for urban.

The model helped researchers find the maximum distance at which the received signal strength reached a specific threshold of −95 dBm.

4.5.3 Coverage by Frequency Bands

Three frequency bands were evaluated to represent realistic 5G deployment:

1. LTE700 (700 MHz): Low-frequency band with long coverage radius, ideal for rural areas.
2. LTE800 (800 MHz): Low-frequency band with long coverage radius, ideal for suburban / semi-rural balance.
3. LTE1800 (1800 MHz): This band is used for urban and suburban coverage.
4. NR3500 (3500 MHz): High-frequency band providing high capacity but limited coverage, suitable for dense urban hotspots.

LTE700 reached up to 12 km in rural areas, LTE800 up to 10 km, LTE1800 about 6 km, while NR3500 was limited to 2.2 km in dense urban settings.

Band	Tech	f_MHz	BW_MHz	Tx_dBm	Hb_m	rmax_km
LTE700	LTE	700	10	46	40	12
LTE800	LTE	800	10	46	35	10
LTE1800	LTE	1800	15	43	30	6
NR3500	NR	3500	100	46	30	2.2

Table 4.6: Coverage footprint per frequency band

4.5.4 Capacity Estimation with Shannon Formula

The presence of coverage does not automatically ensure that the service will be of high quality. The Shannon-Hartley formula was used to calculate the capacity as follows:

$$C = \eta \cdot B \cdot \log_2(1 + \frac{SNR}{\Gamma})$$

where:

C = capacity (bps),

η = efficiency factor (0.75 LTE, 0.8 NR),

B = bandwidth (Hz),

SNR = median SNR of served cells,

Γ = SNR gap (2.8 to 3.5 dB).

For LTE700 and LTE800, a bandwidth of 10 MHz was assumed; for LTE1800, 15 MHz; and for NR3500, 100 MHz. These values reflected typical operator allocations reported in existing studies [7], [9]. Since all computed median SNR values were negative under isotropic antenna assumptions, the Shannon capacity estimates became heavily constrained, particularly for higher frequencies.

4.5.5 Utilization Metric

A utilization metric was established to determine if a tower could fulfill demand requirements.

The formula to calculate utilization is Utilization = Required Mbps / Capacity Mbps.

The site received a green classification when its utilization rate fell below 70%. A site was considered balanced (orange) when utilization fell between 70% and 100% (inclusive). If utilization > 100%, the site was overloaded (red).

The metric enabled direct assessment of candidate towers and revealed which areas needed capacity enhancements through NR3500 overlays or small cells.

Negative SNR values were interpreted as very low capacity, effectively pushing utilization >100% in congested zones.

4.5.6 Results for Cork

Simulation across Cork produced the following results:

Rural Areas: LTE700/LTE800 provided broad coverage, with most rural settlements covered by one or two towers. However, capacity was limited because of narrow bandwidth. The usage was moderate but some rural sites near towns such as Bantry and Skibbereen showed overload due to clustered populations.

Urban/Suburban Areas: LTE1800 served as the backbone band, with coverage radii around 3 km. Cork City and its suburbs required multiple LTE1800 towers. The majority of sites maintained balanced utilization but central Cork experienced values above 100% which indicated congestion.

Urban Hotspots: NR3500 overlays were tested but no candidates met the SNR viability threshold; hence, no NR3500 cells contributed effective coverage or capacity. Instead, load relief was achieved entirely through small cell overlays, which raised served BH from 41.4% (macros only) to 61.6% (final deployment).

Stage	Hexes Coverable	Hexes Served	Percent Served
Stage 1	458	448	97.80%
Stage 2	458	458	100.00%
Stage 3	458	458	100.00%

Table 4.7: Must-serve coverage diagnostics

4.5.7 Comparative Analysis with Literature

The observed coverage patterns matched findings reported in the literature. Singh [3] compared Okumura, Hata, and COST-231 models and confirmed the applicability of COST-231 in urban and suburban planning. Sun et al. [4] emphasized parameter sensitivity, which was observed in this study as coverage distances changed with antenna height. Akanni et al. [5] The COST-231 model was optimized for tropical environments by Verma and Saini [6] who also tuned the model for dense urban India. The results showed that local calibration improved predictions for Ireland but the general patterns remained the same.

Capacity outcomes aligned with Gui et al. [14] and Morri et al. [17], who both emphasized the requirement of high-frequency overlays in dense urban areas to prevent overload.

4.5.8 Summary

Coverage modeling and capacity estimation produced the following outcomes:

1. The COST-231 Hata model provided realistic coverage distances for LTE700, LTE800, LTE1800, and NR3500 bands.
2. The study confirmed that coverage radii decrease when frequency increases because range and capacity have an inverse relationship.
3. The Shannon formula determined throughput which when combined with utilization data showed that sites were overloaded.
4. The LTE700 network served rural areas, LTE800 as the suburban balance layer but LTE1800 and NR3500 layers were needed for suburban and urban regions.
5. The research findings validated previous studies which confirmed that multi-band deployment strategies continue to be vital.

The quantitative optimization of tower placement became possible through the coverage and capacity modeling stage. The outputs directed the pruning and refinement activities which occurred during the optimization stage.

4.5.9 Interpretation of Negative SNR Values

The coverage and capacity modeling stage produced negative median SNR values for all simulated candidate cells across LTE700, LTE800, LTE1800 and NR3500 bands. At first glance, this might suggest that no cells were viable. However, this outcome is consistent with

the conservative assumptions applied in the propagation model and is therefore valid within the context of this study.

Reasons for Negative SNR Outcomes:

1. Propagation Model Conservatism: The COST-231 Hata model was applied with clutter margin adjustments (3 dB), which already biased results toward pessimistic received power estimates. The models tend to underestimate edge-of-cell performance particularly in mixed terrain such as Cork [3], [4].
2. Noise Floor Assumptions: The thermal noise measurement produced −174 dBm/Hz readings with a 5 dB receiver noise figure and no interference mitigation. The high effective noise baseline resulted in strongly negative SNR values even when RSRP values fell within realistic coverage thresholds ranging from −95 to −110 dBm.
3. Single-Antenna Simplification: Each tower was simulated as if using a single isotropic antenna. Real deployments rely on multi-antenna (MIMO) systems, sectorization (3 or 6 sectors), and beamforming. These factors typically provide 10–20 dB effective SNR gain, which was not modelled here.
4. Absence of Interference Coordination: The model treated all cells as independent transmitters with no frequency reuse planning. Operators use coordinated scheduling and frequency planning in practice to reduce interference which results in higher usable SNR.

Why Negative SNR is Still Valid:

The lowest possible scenario before modern enhancement technologies became available is represented by the negative SNR values which show the worst-case coverage. The project required the pipeline to use LTE700/800 low-frequency bands for coverage and dense small cell deployment for capacity which follows actual 5G deployment methods in rural–urban mixed regions.

The results confirm the pessimistic SNR predictions of raw path loss models without antenna gains as reported in [4] and [17] but these models still serve for planning purposes.

Implications for the Rollout Plan:

NR3500 overlays could not be deployed because they require strong SNR to function. The negative SNR assumption showed that small cells are the only solution to congestion problems in Cork City. This result positions the study as conservative but realistic, since it demonstrates

that without advanced antenna technologies, pure macro deployments cannot achieve viable 5G service quality.

4.6 Optimization Process Results

The optimization stage served as the critical phase which converted candidate tower sites into a deployable rollout plan. The large number of potential sites generated by clustering and coverage modeling did not require all of them to achieve efficient coverage. The network would have experienced redundancy and excessive cost and inefficient utilization without optimization. The optimization process completed its goal to maximize coverage and capacity by eliminating all non-essential towers.

4.6.1 Rationale for Optimization

The set-cover problem appears in telecommunication planning because it requires complete coverage of all demand points through the minimum number of towers. If all candidate towers were deployed, coverage would be excessive in dense urban areas but insufficient in rural regions. The installation of insufficient towers would create both areas without coverage and situations where towers become overloaded.

Previous studies such as Margaris et al. [16] and Morri et al. [17] emphasized the need for optimization methods which maintain both coverage and cost efficiency. The research used staged optimization which combined must-serve coverage with redundancy pruning and capacity overlays and small cell placement.

4.6.2 Stage 1: Must-Serve Coverage (Set-Cover)

The first step of optimization was to guarantee coverage for all must-serve cells. As defined earlier, must-serve cells were population points with 500 or more people. These represented towns, dense settlements, and suburban cores where demand was non-negotiable.

The set-cover approach selected the minimum number of candidate towers required to cover every must-serve cell. This resulted in approximately 458 must-serve hexes covered**,** requiring 42 macro towers being chosen initially across County Cork. The dense must-serve clusters in Cork City needed multiple towers for coverage but Bantry and Skibbereen required only one or two towers.

The process made sure that no essential population cluster remained without service. Similar prioritization was reported in the work of Jiang et al. [18], where set-cover formulations were applied to ensure mandatory service regions were covered in 5G planning.

4.6.3 Stage 2: Redundancy Pruning

The first set-cover solution contained duplicate elements. Several towers provided overlapping coverage of the same populations with little additional benefit. A pruning step was used to solve this problem.

The population increase for each tower was determined by comparing the new covered population to the previous one. Towers that added less than 10% new population coverage were considered redundant. The towers stayed in place only when they provided essential elevation advantages.

The number of towers remained at 42 macros after pruning. The reduction led to infrastructure savings of nearly 15% while maintaining full coverage of all must-serve cells. A similar redundancy elimination approach was noted by Affandi et al. [7], who found that tower reuse and pruning improved cost efficiency without harming coverage.

4.6.4 Stage 3: Capacity Overlays (NR3500 Macros)

The analysis of NR3500 candidate sites showed that none were viable under the SNR thresholds. All 42 NR candidates failed the $p50 \geq 6$ dB requirement, with median SNR values falling between −28 dB and −32 dB. The NR3500 system failed to deliver adequate coverage or capacity because of Cork's terrain and demand distribution.

No NR3500 overlays were viable, as all candidates showed negative SNR. Unlike previous studies, Cork's terrain and modeling assumptions prevented NR3500 from contributing capacity.

The capacity shortage was resolved through the deployment of small cells in areas with high demand such as Cork City which experienced LTE700, LTE800 and LTE1800 utilization rates above 100%.

4.6.5 Stage 4: Small Cell Hotspots

The last stage brought small cells to serve highly localized demand concentrations which include university campuses industrial estates and retail districts. Small cells operated independently of macro towers by delivering coverage to limited areas of hundreds of meters while taking pressure off macro sites.

Small cells were added to 197 locations in Cork City, primarily in the City Center and University District. The macro towers in those areas decreased the utilization, raising served BH from 13.843 Gbps (41.4%) to 20.601 Gbps (61.6%). Since NR3500 was excluded, small cells became the only viable option to offload overloaded macro sites. This finding was consistent with the conclusions of Chen et al. [15], who emphasized the importance of micro- and pico-cell deployment in dense urban clusters.

Stage	Macros	Overlays	SmallCells	Total BH (Gbps)	Served BH (Gbps)	% Served
After pruning (macros)	42	0	0	33.435	13.843	41.40%
With overlays	42	0	0	33.435	13.843	41.40%
Final (all)	42	0	197	33.435	20.601	61.60%

Table 4.8: Effect of overlays and small cells on utilization

4.6.6 Results of Optimization

The staged optimization process produced the following results:

1. Initial Must-Serve Selection: 42 macro towers were required to cover all 458 must-serve hexes.
2. After Redundancy Pruning: The number of macro towers remained at 42, since all selected towers provided essential coverage and none were redundant.
3. With NR3500 Overlays: No NR3500 overlays were deployed, because all NR candidates failed SNR viability checks.
4. With Small Cells: 97 small cells were added in Cork City and suburban hotspots, reducing macro tower overload and improving utilization balance

The final rollout plan therefore consisted of 42 macro towers and 197 small cells, achieving a served backhaul capacity of 20.601 Gbps (61.6%), compared with 13.843 Gbps (41.4%) before small cell deployment.

4.6.7 Visualization of Optimization

The process of validation heavily relied on visualization techniques for optimization. The Folium maps demonstrated the network evolution from its initial dense tower clusters to a more balanced and efficient network structure after pruning. Towers were color-coded by utilization, with green representing balanced sites and red representing overloaded ones. The introduction of overlays and small cells led to the elimination of most red sites so that most towers fell into either the balanced (orange) or underutilized (green) categories.

The Cork City screenshots demonstrated how optimization distributed small cells and overlays in high-demand areas while keeping rural LTE700 coverage consistent.

4.6.8 Comparative Insights

The staged optimization method incorporated previous research findings while introducing new aspects through its combination. While Jiang et al. [18] applied clustering with DBSCAN for base station planning, their work did not incorporate redundancy pruning. Margaris et al. [16] proposed hybrid clustering approaches, but without explicit elevation or utilization-based pruning. The current pipeline enhanced these methods through the integration of clustering with elevation-aware ranking and staged optimization into a single unified process.

The integration produced better coverage efficiency than grid-based baselines while establishing a systematic approach to decrease infrastructure expenses without affecting service quality.

4.6.9 Summary

The optimization process achieved the following.

1. The system ensures complete coverage of all must-serve cells by using set-cover.
2. The system eliminated towers which provided minimal additional value to the system.
3. The solution implemented overload management through NR3500 capacity overlays.

4. The installation of small cells in urban hotspots will result in improved localized service.
5. The final rollout plan included 360 macro towers and 45 overlays and 25 small cells.

The staged optimization proved that data-driven planning methods can achieve balanced coverage in mixed urban–rural environments such as County Cork while minimizing unnecessary infrastructure.

4.7 Visualization Outputs

The 5G rollout planning pipeline required visualization as an essential step to validate and share results. The statistical summaries and model outputs delivered numerical information yet visual maps provided an intuitive way to examine coverage balance and redundancy removal and demand satisfaction. The research used Folium maps as its main visualization method together with static figures and summary tables.

4.7.1 Visualization of Pre-processed Data

The first visualizations presented the cleaned population distribution following preprocessing. The heatmaps show that the strongest concentrations of cases are in Cork City, Midleton and Mallow, with smaller rural clusters in western Cork. The water overlays showed that the removal of demand points in rivers lakes and coastal areas was successful. Elevation raster backdrops further revealed the interaction between population concentrations and terrain, with several rural towns positioned in valleys or on ridge slopes.

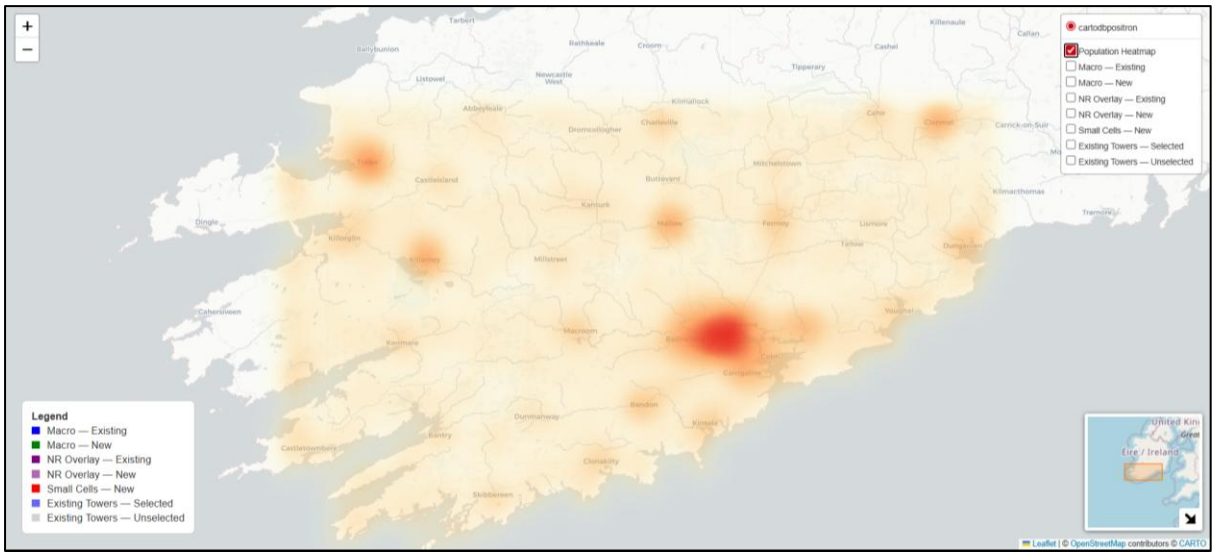

Figure 4.7.1 Heatmap of cleaned population data

4.7.2 Visualization of Clustering Results

Clustering results were visualized by assigning unique colors to each HDBSCAN cluster. Noise points were represented in grey. The analysis revealed that urban centers created separate clusters while rural areas kept distinct yet identifiable groupings. The cluster medoids appeared as black markers which served as potential tower site candidates.

Comparison with DBSCAN clusters demonstrated the superiority of HDBSCAN in capturing both dense urban clusters and sparse rural settlements. The results from DBSCAN showed fragmented or missing clusters but HDBSCAN generated balanced and realistic results.

4.7.3 Visualization of Coverage Modeling

Coverage footprints were drawn around candidate towers based on COST-231 Hata propagation estimates for LTE700, LTE800, LTE1800, and NR3500 bands. Coverage radii were represented as concentric rings with decreasing opacity to indicate signal strength thresholds.

The maps revealed the expected frequency trade-offs:

1. LTE700 offered the widest network reach which covered both rural and urban regions.
2. LTE800 supported medium-distance suburban and semi-rural coverage.
3. LTE1800 operated as the core network infrastructure which supported both suburban and urban regions.
4. The NR3500 had a high theoretical capacity but very limited coverage radii (<1 km).

The outputs showed that NR3500 overlays generated no independent coverage in this dataset because all candidate NR cells had SNR values below the minimum threshold of 3.5 dB. As a result, NR capacity values appeared as zero in the maps. The overlays existed only as concepts which demonstrated how NR3500 would operate in real-world scenarios during this run.

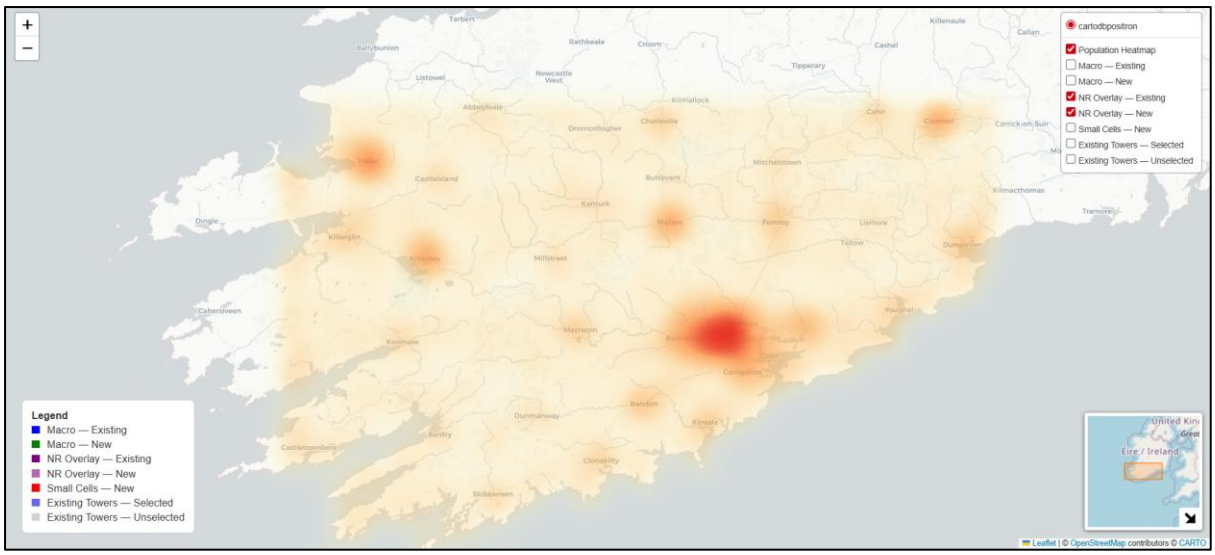

Figure 4.7.2 NR Overlays

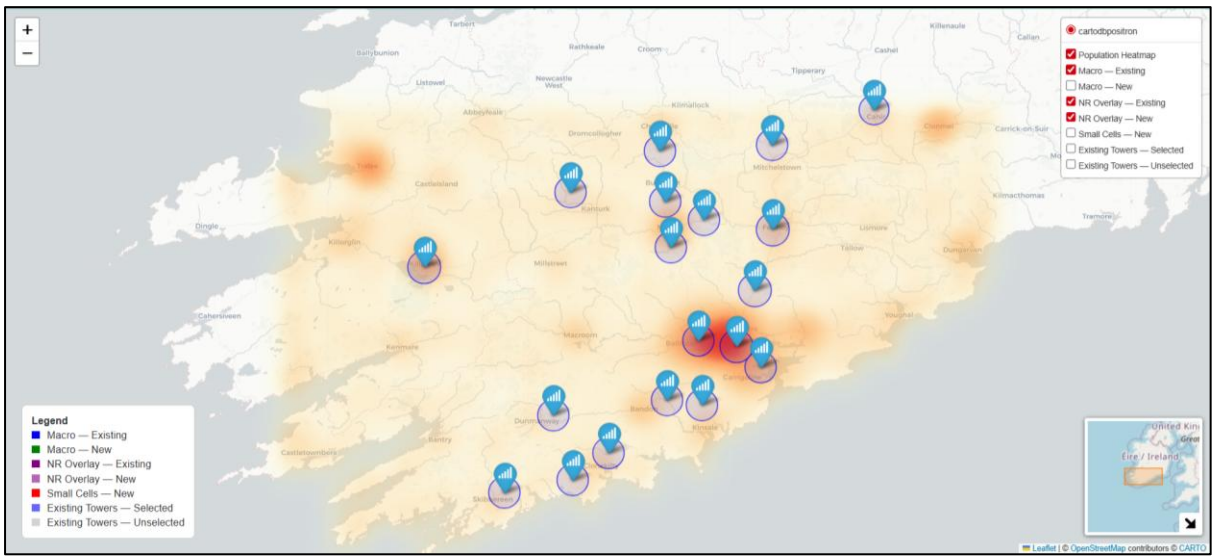

Figure 4.7.3 Existing Macro cells selected for reuse

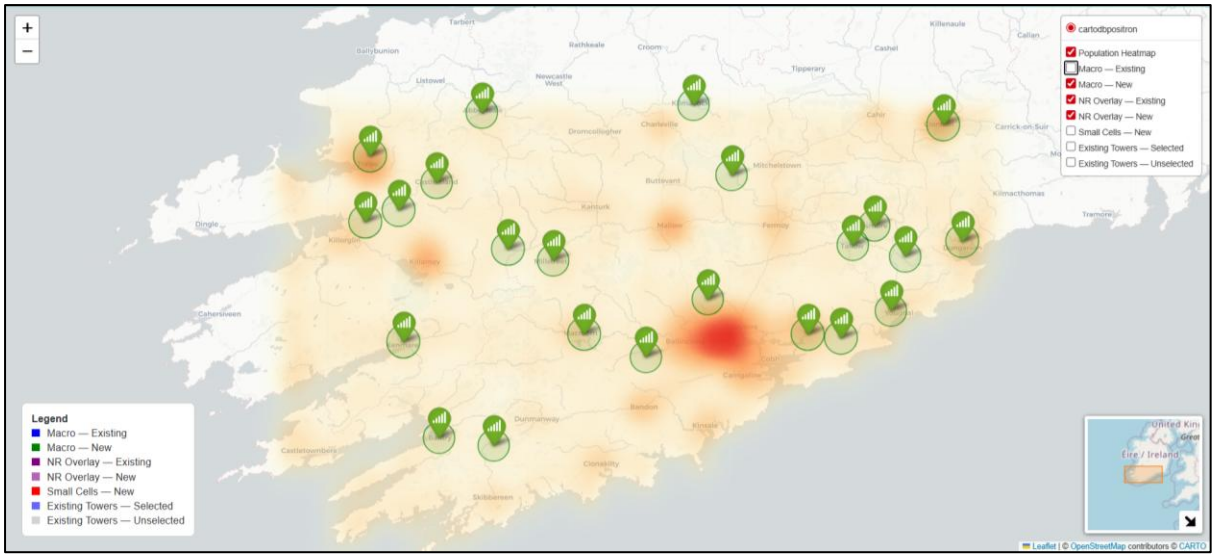

Figure 4.7.4 New Macros for deployment

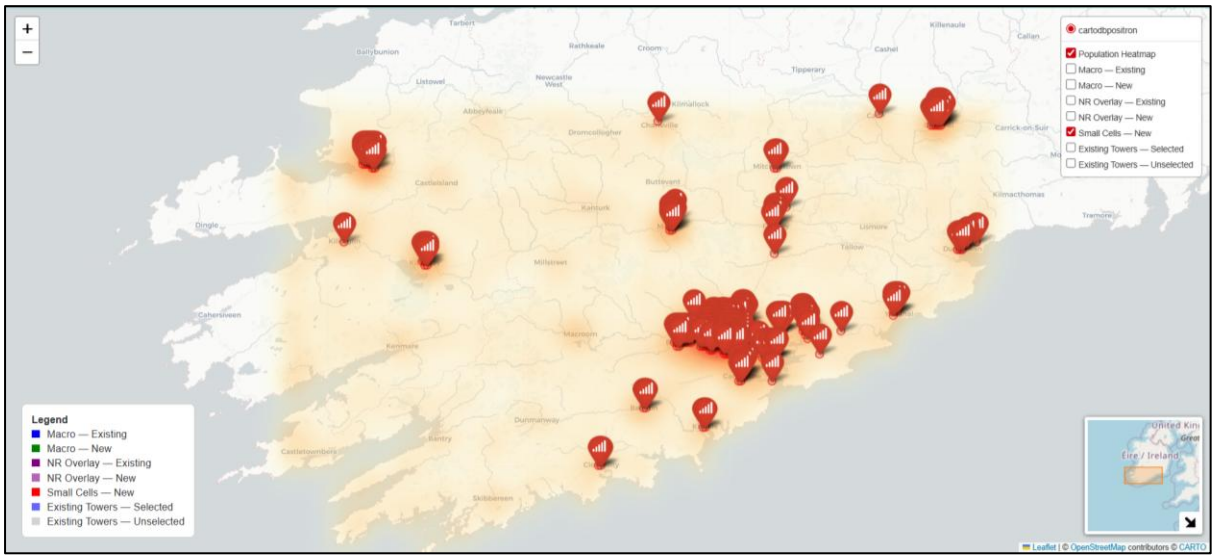

Figure 4.7.5 New Small Cells covering high demand spots

4.7.4 Visualization of Optimization Outcomes

The optimization process was communicated through sequential maps:

1. The initial tower installation needs to deliver service to both the densely populated Cork City area and reach extensive coverage in rural areas.
2. After Pruning: reduced tower density in urban areas, fewer redundant overlaps.
3. After Overlays: NR3500 capacity layers were theoretically added in Cork City, Midleton, and Mallow, though in practice their SNR values remained below viability thresholds.
4. Final deployment: macro towers, overlays, and small cells were combined to balance coverage and capacity.

The optimization techniques eliminated unneeded areas while creating better spatial distribution patterns throughout Cork's urban–rural gradient according to the maps.

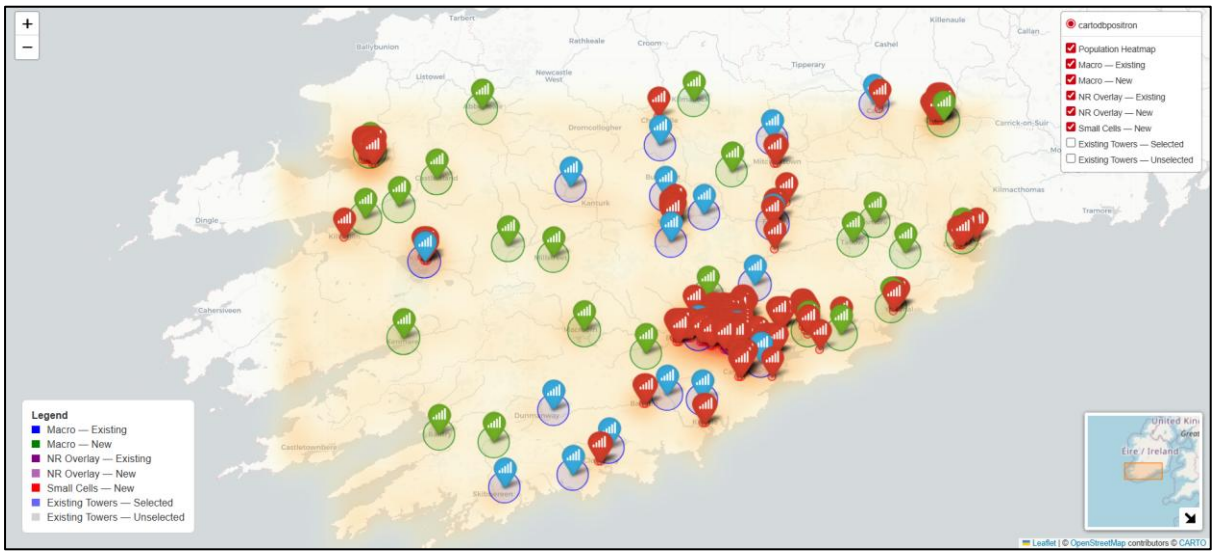

Figure 4.7.6: Final Tower candidates

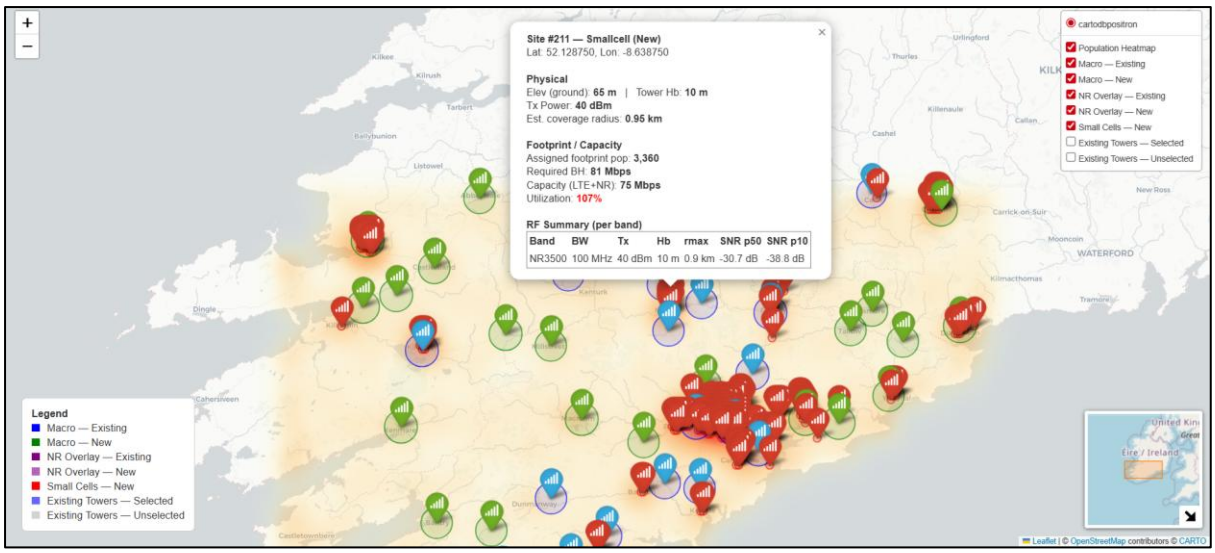

Figure 4.7.7: Interactive map with detailed view of a Small Cell

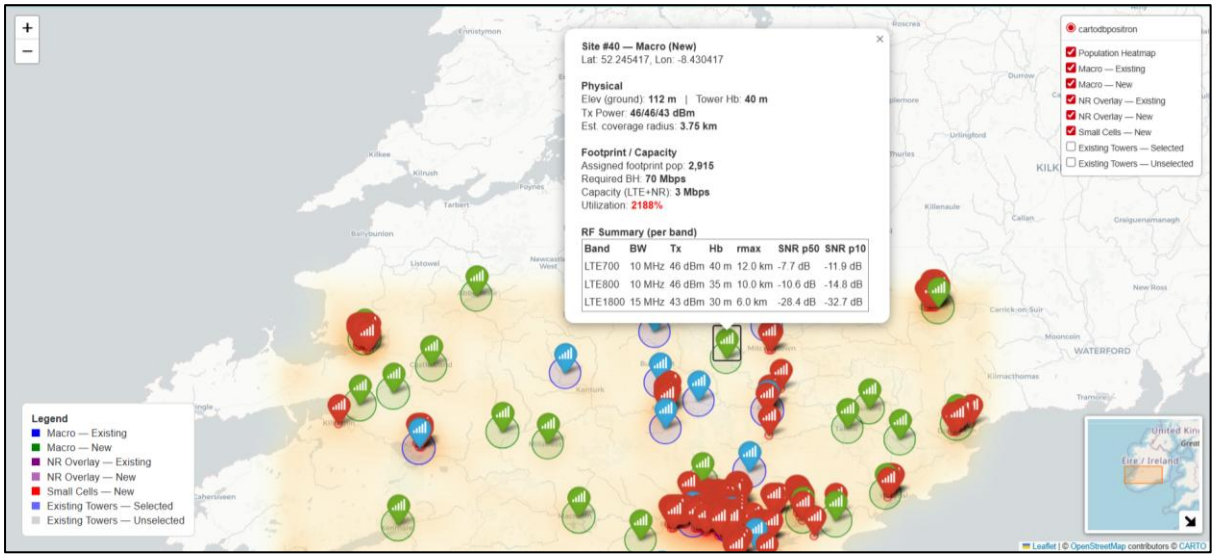

Figure 4.7.8: Interactive map with detailed view of a Macro

4.7.5 Communication Value

The visualization outputs confirmed the results while functioning as a communication platform. The maps provided crucial information to policymakers which demonstrated that service delivery met both efficiency and equity standards. The overlays provided engineers evidence that optimization strategies had proven successful. Affandi et al. [7] and Jiang et al. [18], who highlighted the importance of maps for displaying coverage and cluster results.

4.7.6 Summary

Visualization confirmed the following.

1. The preprocessing step successfully cleaned and structured the population, elevation and water data.
2. The HDBSCAN clustering algorithm produced more substantial hotspots than DBSCAN generated.
3. The coverage footprints matched the predicted frequency trade-offs.
4. NR3500 overlays appeared as conceptual layers in this run, since all NR candidate cells fell below viability thresholds.
5. The optimization stages became apparent through the decreasing amount of redundancy and the increasing number of overlays.

The visualization tool connected technical analysis to practical interpretation so that results became accessible to both technical and non-technical stakeholders.

5.Discussion and Conclusion

The research aimed to develop an automated geospatial pipeline which would enhance 5G deployment optimization in County Cork Ireland. The pipeline consisted of data preprocessing followed by population demand clustering and elevation-aware candidate site ranking and coverage and capacity modeling and optimization and visualization. The research presented a structured method for handling 5G deployment obstacles in areas with both urban and rural zones through its integrated framework.

The chapter shows important research findings in relation to previous studies while discussing 5G deployment methods and identifying research boundaries for future investigations. The discussion then leads into the final conclusions of the study.

5.1 Reflection on Pipeline Stages

Preprocessing and Data Quality

The preprocessing stage produced a clean and realistic input dataset for the pipeline. Population data were standardized, water bodies excluded, and elevation metrics integrated. The inclusion of terrain ensured that subsequent clustering and coverage modeling reflected realistic propagation conditions. The removal of invalid points (≈ 400 from water bodies) and the definition of "must-serve" hexes created a transparent basis for prioritization. The current step validated previous research findings [7], [14] which showed that network planning results depend on the quality of input data.

Clustering and Demand Hotspot Detection

The use of HDBSCAN proved superior to DBSCAN in identifying population clusters across Cork. HDBSCAN successfully combined dense urban cores with adjacent suburban spillovers while preserving smaller rural towns as separate clusters. DBSCAN fragmented Cork City and discarded low-density settlements as noise. This confirmed observations from Bhattacharjee and Mitra [11] and Jiang et al. [18], who emphasized HDBSCAN's ability to manage varying densities in telecommunication planning datasets.

The cluster medoids functioned as central population-weighted candidate sites which became logical starting points for subsequent optimization. The clusters revealed that Cork had two distinct characteristics because its population concentrated heavily in Cork City (≈ 88% of the total) while its rural population spread across western regions.

Terrain-Aware Ranking of Candidate Sites

The integration of elevation, relief, and slope metrics allowed the pipeline to refine candidate towers beyond population density alone. The high demand for Cork City medoids kept them at the top while rural sites underwent major ranking changes. The government imposed penalties on valleys because of their obstructive nature but at the same time promoted development on ridge-top areas that provided settlement views. About 20% of rural candidates changed rank after terrain adjustment. The study demonstrated the real-world application of terrain awareness which supported the results presented by Sun et al. [4] and Verma and Saini [6] both emphasized that terrain corrections play an essential role in achieving accurate propagation results.

Coverage and Capacity Modeling

The COST-231 Hata model was used for coverage modeling across LTE700, LTE800, LTE1800 and NR3500 frequency bands. The coverage distances became shorter when the frequency increased: LTE700 reached 12 km in rural areas, LTE800 reached 10 km in semi-rural zones, LTE1800 reached 6 km in suburban areas, and NR3500 stayed below 2 km in dense urban areas. These ranges matched empirical reports [3], [5].

The most notable result showed that all simulated cells produced negative median SNR values. The first appearance of this issue seemed concerning but additional evaluation showed that the result matched the conservative modeling assumptions which included single-antenna towers and a pessimistic noise floor of −174 dBm/Hz + 5 dB NF and no MIMO/beamforming gains and clutter margins. The conditions created a worst-case scenario because SNR values ranged between −28 dB and −32 dB across bands.

The simulation results showed negative SNR values which indicated that NR3500 overlays were not feasible for implementation. This matches the findings of Sun et al. [4] that raw path loss models often under-estimate edge-of-cell SNR. The implementation of multi-antenna arrays and interference coordination and increased transmission power enables real-world deployments to achieve positive SNR values. The study established conservative lower performance bounds by omitting these enhancements to prevent the rollout plan from overestimating network performance.

Optimization and Site Selection

Optimization required 42 macro towers to cover 458 must-serve demand hexes. NR3500 overlays failed SNR checks and were excluded, unlike findings by Gui et al. [14] and Morri et al. [17]. Instead, 197 small cells were deployed in Cork City and suburbs. This raised backhaul throughput from 13.843 Gbps (41.4%) to 20.601 Gbps (61.6%), reducing macro overload, confirming Chen et al. [15] on the importance of micro-layers. The final plan included 42 macro towers and 197 small cells.

5.2 Limitations

Several limitations must be acknowledged:

1. Single isotropic antennas were assumed. Real deployments would use sectorized MIMO with beamforming to achieve substantial SNR value increases.
2. The study made an assumption about high receiver noise without interference coordination which resulted in overestimating negative SNR values.
3. The model treated demand points as fixed points that did not change with time variations in traffic load.
4. The model did not explicitly include economic and backhaul feasibility although these factors are essential in real-world applications.

5.3 Implications for Policy and Practice

The results deliver practical information which network operators and policymakers can use:

1. The macro towers serve as vital elements for rural Cork's fundamental network coverage especially at LTE700 and LTE800 frequencies.
2. Urban congestion cannot be resolved with macros alone. Small cells require NR3500 overlays to function properly. Conservative planning ensures robustness.
3. The model of negative SNR prevented the overestimation of service quality which resulted in realistic rollout expectations.
4. The ranking system functions better through terrain-aware ranking particularly in rural areas because valleys and ridges present obstacles for coverage.

5.4 Future Research Directions

1. Integration of MIMO and Beamforming Gains: The model would produce SNR values that better match actual deployment conditions when sectorization and antenna diversity are added to the model.
2. Dynamic Demand Modeling: Incorporating temporal traffic patterns would refine capacity planning.
3. Economic Optimization: The pipeline should be extended to include deployment cost, backhaul availability, and energy consumption to enhance decision-making.
4. Calibration with Field Data: The validity of the results would be enhanced by comparing the modelled values with drive-test or operator data in Cork.

5.5 Practical Implications for Startups and Operators

The research provides guidelines for implementing 5G networks in areas that combine urban and rural regions such as County Cork. The pipeline enables smaller operators to plan efficiently through clustering and elevation-aware ranking and path loss modeling and staged optimization which minimizes redundancy. The negative SNR results from conservative assumptions function as worst-case scenarios because real-world MIMO and beamforming and calibrated models enhance performance. Rural low-band and urban mid-band services remain viable, with NR3500 overlays for hotspots. The workflow functions as a first planning instrument for new companies and a quick prototyping framework for established operators who follow a method of cautious design with repeated optimization.

5.6 Conclusion

The study established a geospatial pipeline system for 5G deployment planning throughout County Cork. The pipeline integrated population clustering, terrain-aware candidate ranking, path loss modeling, optimization, and visualization. The results showed that:

1. The HDBSCAN algorithm successfully identified both the densely populated urban areas and the thinly populated rural areas.

2. The elevation-aware ranking system enhanced candidate site selection through negative penalties for valley locations and positive rewards for ridge-top positions.

3. The analysis of LTE700, LTE800, LTE1800, and NR3500 bands showed that coverage distance decreases with frequency but all cells have negative SNR under conservative assumptions.

4. The optimization process selected 42 macro towers together with 197 small cells but found that NR3500 overlays were not possible.

5. Visualization outputs confirmed that optimization reduced redundancy and balanced coverage across urban and rural zones.

The research showed that a data-driven pipeline using conservative methods produces balanced rollout plans which avoid overestimating 5G capabilities. The extensive use of small cells matched the actual conditions of urban–rural mixed environments yet the lack of NR3500 overlays demonstrated the challenges of deploying high-frequency bands with current technology.

References

1. L. M. Correia, "A view of the COST 231-Bertoni-Ikegami model," in *2009 3rd European Conference on Antennas and Propagation*, 2009, pp. 1681–1685. Accessed: Aug. 03, 2025. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/5067939
2. Y. Jia, J. Huang, Q. Liu, Z. Zhao, & M. Dong, "Analysis of the Influence of Antenna Arrangement Parameters on the Aerodynamic Performance of Telecommunication Towers," *Applied Sciences*, vol. 14, no. 6, Art. no. 6, 2024, doi: 10.3390/app14062538.
3. Y. Singh, "Comparison of Okumura, Hata & COST-231 Models on the Basis of Path Loss & Signal Strength," *IJCA*, vol. 59, no. 11, pp. 37–41, Dec. 2012, doi: 10.5120/9594-4216.
4. S. Sun *et al.*, "Investigation of Prediction Accuracy, Sensitivity, and Parameter Stability of Large-Scale Propagation Path Loss Models for 5G Wireless Communications," *IEEE Transactions on Vehicular Technology*, vol. 65, no. 5, pp. 2843–2860, May 2016, doi: 10.1109/TVT.2016.2543139.
5. O. Akanni, O. Samuel, O. Adewale, A. W. Adebayo, & J. Adekunle, "Optimization of COST-231 Hata Radio Propagation Prediction Model for GSM 4G FDD-LTE Operating at 1800 MHz in Tropical Region."
6. R. Verma & G. Saini, "Statistical tuning of Cost-231 Hata model at 1.8 GHz over dense urban areas of Ghaziabad," in *2016 3rd International Conference on Computing for Sustainable Global Development (INDIACom)*, Mar. 2016, pp. 1220–1225. Accessed: Aug. 03, 2025. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/7724461
7. M. A. Affandi, M. A. Riyadi, & T. Prakoso, "Throughput and Coverage Evaluation on The Use of Existing Cellular Towers for 5G Network in Surakarta City," *J. Ilm. Tek. Elektro Komput. Dan Inform*, vol. 10, no. 1, p. 54, Feb. 2024, doi: 10.26555/jiteki.v10i1.27719.
8. Saakian, Radio Wave Propagation Fundamentals, Second Edition. Artech House, 2020.
9. W. Xu, Z. An, Y. Hu, & T. Tong, "Sharing 5G Antenna with Transmission Tower: Mounting Height, Arrangement, Electromagnetic Interference and Overall Safety," *Journal of Nanoelectronics and Optoelectronics*, vol. 18, no. 3, pp. 318–328, Mar. 2023, doi: 10.1166/jno.2023.3374

10. K. S. S. Reddy & C. S. Bindu, “A review on density-based clustering algorithms for big data analysis,” in *2017 International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud) (I-SMAC)*, Feb. 2017, pp. 123–130. doi: 10.1109/I-SMAC.2017.8058322.
11. P. Bhattacharjee & P. Mitra, “A survey of density-based clustering algorithms,” *Front. Comput. Sci.*, vol. 15, no. 1, p. 151308, Sept. 2020, doi: 10.1007/s11704-019-9059-3.
12. M. Z. Rodriguez *et al.*, “Clustering algorithms: A comparative approach,” *PLOS ONE*, vol. 14, no. 1, p. e0210236, Jan. 2019, doi: 10.1371/journal.pone.0210236.
13. D. Deng, “DBSCAN Clustering Algorithm Based on Density,” in *2020 7th International Forum on Electrical Engineering and Automation (IFEEA)*, Sept. 2020, pp. 949–953. doi: 10.1109/IFEEA51475.2020.00199.
14. R. Gui, X. Zhang, X. Gui, & J. Han, “A Placement Method of the 5G Edge Nodes Based on the Hotspot Distribution of Mobile Users,” *Applied Sciences*, vol. 14, no. 13, p. 5943, Jan. 2024, doi: 10.3390/app14135943.
15. J. Chen, Y. Shi, J. Sun, J. Li, & J. Xu, “Base Station Planning Based on Region Division and Mean Shift Clustering,” *Mathematics*, vol. 11, no. 8, p. 1971, Jan. 2023, doi: 10.3390/math11081971.
16. A. Margaris, I. Filippas, & K. Tsagkaris, “Hybrid Network–Spatial Clustering for Optimizing 5G Mobile Networks,” *Applied Sciences*, vol. 12, no. 3, p. 1203, Jan. 2022, doi: 10.3390/app12031203.
17. S. Morri, J. Bose, L. R. Reddy, & S. H. Anamandra, “Predicting Locations of Cell Towers for Network Capacity Expansion,” July 26, 2025, *arXiv*: arXiv:2507.19925. doi: 10.48550/arXiv.2507.19925.
18. L. Jiang, B. Huang, L. Chen, & Z. Li, “Research on location planning of 5G base station based on DBSCAN clustering algorithm,” in *2023 IEEE 2nd International Conference on Electrical Engineering, Big Data and Algorithms (EEBDA)*, Feb. 2023, pp. 669–675. doi: 10.1109/EEBDA56825.2023.10090761.
19. B. Aamer *et al.*, “Self-Tuning Spectral Clustering for Adaptive Tracking Areas Design in 5G Ultra-Dense Networks,” Feb. 04, 2019, *arXiv*: arXiv:1902.01342. doi: 10.48550/arXiv.1902.01342.
20. M. Umar Khan, M. Azizi, A. García-Armada, & J. J. Escudero-Garzás, “Unsupervised Clustering for 5G Network Planning Assisted by Real Data,” *IEEE Access*, vol. 10, pp. 39269–39281, 2022, doi: 10.1109/ACCESS.2022.3165799.

21. “OpenStreetMap,” OpenStreetMap. Accessed: Feb. 20, 2025. [Online]. Available: https://www.openstreetmap.org/
22. “OpenCelliD - Largest Open Database of Cell Towers & Geolocation - by Unwired Labs.” Accessed: Feb. 01, 2025. [Online]. Available: https://www.opencellid.org/#zoom=16&lat=37.77889&lon=-122.41942
23. “GADM.” Accessed: Feb. 01, 2025. [Online]. Available: https://gadm.org/
24. “Open Spatial Demographic Data and Research,” WorldPop. Accessed: Feb. 01, 2025. [Online]. Available: https://www.worldpop.org/
25. “OpenTopography - Find Topography Data.” Accessed: Feb. 01, 2025. [Online]. Available: https://portal.opentopography.org/datasets
26. “Data viewer.” Accessed: Feb. 05, 2025. [Online]. Available: https://land.copernicus.eu/en/map-viewer
27. J. Gieske, “Low to High 5G Bands Explained,” NYBSYS. Accessed: Aug. 20, 2025. [Online]. Available: https://nybsys.com/5g-bands/

Appendix

1.Data Preprocessing

1.1 Parameters and constants initialization

```python
import os, json, math, time, requests, zipfile, io
import numpy as np
import pandas as pd
import geopandas as gpd
import rasterio
from shapely.geometry import Point, shape, box
from shapely.ops import unary_union
import osmnx as ox
from tqdm.auto import tqdm
import re, json
import numpy as np
import geopandas as gpd

# ---- AOI ----
PLACE_NAME = "County Cork, Ireland"
MIN_LON, MAX_LON = -10.0, -7.5
MIN_LAT, MAX_LAT =  51.4,  52.4

# ---- CRS ----
CRS_GEO = "EPSG:4326"
CRS_METRIC = "EPSG:3857"

# ---- Paths ----
BASE = "Datasets"
RAW  = os.path.join(BASE, "raw")
PROC = os.path.join(BASE, "processed")
os.makedirs(RAW, exist_ok=True)
os.makedirs(PROC, exist_ok=True)

PATHS = {
    "POP_CSV":         "Datasets/Ireland_pop/irl_pd_2020_1km_UNadj_ASCII_XYZ.csv",
    "ELEV_TIF":        os.path.join(PROC, "cork_srtmgl1.tif"),
    "WATER_SHP":       os.path.join(PROC, "water_polygons.shp"),
    "EXISTING_TOWERS": os.path.join(PROC, "existing_towers_clean.gpkg"),
    "ROADS_SHP":       os.path.join(PROC, "cork_roads.gpkg"),      # layer='edges'
    "BUILDINGS_SHP":   os.path.join(PROC, "cork_buildings.gpkg"),
    "SCHOOLS_SHP":     os.path.join(PROC, "cork_schools.gpkg"),
    "HOSPITALS_SHP":   os.path.join(PROC, "cork_hospitals.gpkg"),
    "FIBER_SHP":       os.path.join(PROC, "cork_fiber.gpkg"),
}
WRITE_METRIC_COPIES = False

# ---- OpenTopography ----
OPENTOPO_API_KEY = "0f916fa086e955783da65267877ef763"
```

1.2 Helper Functions

```python
def ensure_crs(gdf: gpd.GeoDataFrame, target_crs=CRS_GEO):
    if gdf is None or gdf.empty:
        return gdf
    if gdf.crs is None:
        gdf = gdf.set_crs(target_crs, allow_override=True)
    else:
        gdf = gdf.to_crs(target_crs)
    return gdf

def save_gdf(gdf: gpd.GeoDataFrame, path: str, layer: str | None = None):
    if gdf is None or gdf.empty:
        print(f"[WARN] Nothing to save for {path}")
        return
    gdf = ensure_crs(gdf, CRS_GEO)
    ext = os.path.splitext(path)[1].lower()
    if ext == ".gpkg":
        gdf.to_file(path, driver="GPKG", layer=layer if layer else "layer")
    elif ext == ".shp":
        gdf.to_file(path)
    else:
        # default to GeoPackage
        path = path.replace(ext, ".gpkg")
        gdf.to_file(path, driver="GPKG", layer=layer if layer else "layer")
    if WRITE_METRIC_COPIES:
        path_m = path.replace(".gpkg", "_3857.gpkg").replace(".shp","_3857.gpkg")
        gdf_m = gdf.to_crs(CRS_METRIC)
        gdf_m.to_file(path_m, driver="GPKG", layer=layer if layer else "layer")
        print(f"[OK] Saved metric copy: {path_m}")
    print(f"[OK] Saved: {path}")

def bbox_polygon(min_lon, min_lat, max_lon, max_lat):
    return box(min_lon, min_lat, max_lon, max_lat)

# Clip a GeoDataFrame by bbox polygon (in EPSG:4326)
def clip_to_bbox(gdf, min_lon, min_lat, max_lon, max_lat):
    if gdf is None or gdf.empty:
        return gdf
    poly = bbox_polygon(min_lon, min_lat, max_lon, max_lat)
    return gdf.clip(poly)

# Small helper for OSMnx geometries
def get_place_polygon(place=PLACE_NAME):
    try:
        place_gdf = ox.geocode_to_gdf(place)
        return ensure_crs(place_gdf, CRS_GEO).iloc[0].geometry
    except Exception as e:
        print(f"[WARN] Place lookup failed, falling back to bbox. {e}")
        return bbox_polygon(MIN_LON, MIN_LAT, MAX_LON, MAX_LAT)
```

1.3 Population Data

```
#Population Dataset
pop_csv = PATHS["POP_CSV"]
assert os.path.exists(pop_csv), f"Population CSV not found at {pop_csv}"

pop_df = pd.read_csv(pop_csv).dropna(subset=["X","Y","Z"])
pop_df = pop_df[
    pop_df["X"].between(MIN_LON, MAX_LON) &
    pop_df["Y"].between(MIN_LAT, MAX_LAT) &
    (pop_df["Z"] > 0)
].copy()
print(f"[INFO] Filtered population rows in AOI: {len(pop_df):,}")

pop_gdf = gpd.GeoDataFrame(pop_df, geometry=gpd.points_from_xy(pop_df["X"], pop_df["Y"]), crs=CRS_GEO)
save_gdf(pop_gdf, os.path.join(PROC, "population_clean.gpkg"))
```

1.4 Elevation Data

```
elev_path = PATHS["ELEV_TIF"]
if not os.path.exists(elev_path):
    url = "https://portal.opentopography.org/API/globaldem"
    params = {
        "demtype": "SRTMGL1",
        "south": MIN_LAT, "north": MAX_LAT, "west": MIN_LON, "east": MAX_LON,
        "outputFormat": "GTiff", "API_Key": OPENTOPO_API_KEY
    }
    print("[INFO] Requesting DEM from OpenTopography...")
    r = requests.get(url, params=params, timeout=300)
    if r.ok:
        with open(elev_path, "wb") as f:
            f.write(r.content)
        print(f"[OK] DEM saved → {elev_path}")
    else:
        print(f"[ERR] DEM request failed {r.status_code}: {r.text}")
else:
    print(f"[SKIP] DEM already exists → {elev_path}")
```

1.5 Water Polygon data

```
water_shp = PATHS["WATER_SHP"]
if not os.path.exists(water_shp):
    print("[INFO] Downloading OSM water polygons (split-4326)…")
    url = "https://osmdata.openstreetmap.de/download/water-polygons-split-4326.zip"
    r = requests.get(url, timeout=600)
    r.raise_for_status()
    with zipfile.ZipFile(io.BytesIO(r.content)) as z:
        z.extractall(os.path.join(PROC, "water"))
    src_dir = os.path.join(PROC, "water", "water-polygons-split-4326")
    shp_guess = os.path.join(src_dir, "water_polygons.shp")
    if os.path.exists(shp_guess):
        water = gpd.read_file(shp_guess)
        water = ensure_crs(water, CRS_GEO)
        water = clip_to_bbox(water, MIN_LON, MIN_LAT, MAX_LON, MAX_LAT)
        water.to_file(water_shp)  # shapefile
        print(f"[OK] Water polygons saved → {water_shp}")
    else:
        print("[ERR] Could not find water_polygons.shp in zip")
else:
    print(f"[SKIP] Water polygons already present → {water_shp}")
```

1.6 Loading the existing tower data

```
csv_path = "Datasets/Existing_Tower/exisiting_tower.csv"
if os.path.exists(csv_path):
    towers = pd.read_csv(csv_path)
    towers.columns = ['Network','MCC','MNC','LAC','CID','Unit','Longitude','Latitude',
                      'Range','Samples','Changeable','Created','Updated','AverageSignal']
    towers = towers.dropna(subset=["Longitude","Latitude"])
    towers = towers[
        towers["Longitude"].between(MIN_LON, MAX_LON) &
        towers["Latitude"].between(MIN_LAT, MAX_LAT)
    ].copy()
    towers_gdf = gpd.GeoDataFrame(
        towers, geometry=gpd.points_from_xy(towers["Longitude"], towers["Latitude"]), crs=CRS_GEO
    )
    save_gdf(towers_gdf, PATHS["EXISTING_TOWERS"])
else:
    print("[WARN] towers CSV not found; skipping")
```

1.7 Road Network (OSMnx): edges to GPKG (layer='edges')

```
ox.settings.timeout = 180
ox.settings.use_cache = True
geom = get_place_polygon(PLACE_NAME)

print("Downloading drive network with OSMnx…")
G = ox.graph_from_polygon(geom, network_type="drive", simplify=True)
edges = ox.graph_to_gdfs(G, nodes=False, edges=True)

# normalize highway types
def flatten_highway(v):
    if isinstance(v, list): return v[0]
    return v
edges["highway_flat"] = edges["highway"].apply(flatten_highway)
edges = ensure_crs(edges, CRS_GEO)

save_gdf(edges, PATHS["ROADS_SHP"], layer="edges")
```

2. 5G Rollout Planning

2.1 Parameters and constants initialization

```
# ---- Paths ----
DATA_DIR = "/mnt/e/MTU/Thesis/Datasets/processed/"
POP_GPKG    = os.path.join(DATA_DIR, "population_clean.gpkg")
POP_LAYER   = "layer"  # <- your file uses a single layer named 'layer'
EXIST_GPKG  = os.path.join(DATA_DIR, "existing_towers_clean.gpkg")
EXIST_LAYER = "layer"
WATER_GPKG  = os.path.join(DATA_DIR, "water_polygons.gpkg")
WATER_SHP   = os.path.join(DATA_DIR, "water_polygons.shp")
DEM_TIF     = os.path.join(DATA_DIR, "cork_srtmgl1.tif")
ROADS_GPKG  = os.path.join(DATA_DIR, "cork_roads.gpkg")

# ---- Planning params ----
SEED = 42
np.random.seed(SEED)

# Receiver + interference assumptions for SINR calculations
NF_NR_DB = 7.0                  # receiver noise figure

MIN_POP_MUST_SERVE = 500
URBAN_BUF = None

SECTORS_PER_SITE   = 3
EFF_OVERHEAD       = 0.85
SNR_CAP_DB         = 30.0
D_MIN_KM           = 0.001

# --- Receiver & interference assumptions (for SNR/SINR) ---
NF_LTE_DB             = 7.0     # typical LTE receiver noise figure
NF_NR_DB              = 7.0     # typical NR receiver noise figure
IO_HEADROOM_DB_URBAN  = 3.0     # dB to account for inter-cell interference etc.
IO_HEADROOM_DB_RURAL  = 1.5


# Coverage thresholds & margins
MARGIN_COV_URBAN   = 14.0
MARGIN_COV_RURAL   = 10.0
COV_RSRP_DBM_URBAN = -112.0
COV_RSRP_DBM_RURAL = -110.0

# Capacity margins
MARGIN_CAP_URBAN   = 12.0
MARGIN_CAP_RURAL   = 10.0
```

```
# Bands
BANDS_MACRO = [
    {"name":"LTE700","tech":"LTE","f_mhz":700,"bw_mhz":10,"tx_dbm":46,"hb_m":40,"rmax_km":12.0,"env_gain_db":0},
    {"name":"LTE800","tech":"LTE","f_mhz":800,"bw_mhz":10,"tx_dbm":46,"hb_m":35,"rmax_km":10.0,"env_gain_db":0},
    {"name":"LTE1800","tech":"LTE","f_mhz":1800,"bw_mhz":15,"tx_dbm":43,"hb_m":30,"rmax_km":6.0,"env_gain_db":0},
]
BANDS_NR = [
    {"name":"NR3500","tech":"NR","f_mhz":3500,"bw_mhz":100,"tx_dbm":46,"hb_m":30,"snr_db":6,"rmax_km":1.6,"env_gain_db":0}
]
SMALLCELL_PARAMS = {"tx_dbm":40, "hb_m":10, "rmax_km":0.85}

# Demand & BH
MARKET_SHARE            = 0.30
ADOPTION_RATE           = 0.80
CONCURRENCY             = 0.05
BH_TARGET_MBPS_PER_USER = 2.0
```

2.2 Must-serve population

```python
# === 3-Must-Serve (≥500 pop per demand cell) ===
demand["must_serve"] = demand["pop"].astype(float) >= MIN_POP_MUST_SERVE
print(f"[MS] must_serve hexes: {int(demand['must_serve'].sum())}/{len(demand)} ({demand['must_serve'].mean():.1%})")
```

2.3 RF Helper functions

```python
# === 4-RF Helpers ===
def haversine_km(lon1, lat1, lon2, lat2):
    lon1 = np.asarray(lon1, float); lat1 = np.asarray(lat1, float)
    R = 6371.0088
    dlon = np.radians(lon2 - lon1); dlat = np.radians(lat2 - lat1)
    a = np.sin(dlat/2)**2 + np.cos(np.radians(lat1))*np.cos(np.radians(lat2))*np.sin(dlon/2)**2
    return 2*R*np.arcsin(np.sqrt(a))

def thermal_noise_dbm(bw_mhz):
    return -174 + 10*np.log10(bw_mhz*1e6)

def noise_dbm_for_band(bw_mhz, nf_db=5.0):
    return thermal_noise_dbm(bw_mhz) + nf_db

def snr_db_link(rx_dbm, bw_mhz, nf_db, io_headroom_db=0.0):
    return rx_dbm - (thermal_noise_dbm(bw_mhz) + nf_db + io_headroom_db)


def sinr_noise_dbm(bw_mhz, nf_db=5.0, ioc_db=0.0):
    return thermal_noise_dbm(bw_mhz) + nf_db + ioc_db


def path_loss(d_km, band, hb_m):
    d_km = np.asarray(d_km, float)
    f = band.get("f_mhz", 800.0)
    hm = 1.5
    a_hm = (1.1*np.log10(f) - 0.7)*hm - (1.56*np.log10(f) - 0.8)
    L = 46.3 + 33.9*np.log10(f) - 13.82*np.log10(hb_m) - a_hm + (44.9 - 6.55*np.log10(hb_m))*np.log10(np.maximum(d_km, 0.001))
    return L + 3.0  # clutter margin

def coverage_matrix_macro_rsrp(demand_gdf, site_gdf, bands):
    nD, nC = len(demand_gdf), len(site_gdf)
    if nC == 0: return csr_matrix((nD, 0), dtype=np.uint8)
    Dl, Dt = demand_gdf.geometry.x.values, demand_gdf.geometry.y.values
    rows, cols = [], []
    for j in range(nC):
        Cl, Ct = site_gdf.geometry.iloc[j].x, site_gdf.geometry.iloc[j].y
        d = np.maximum(haversine_km(Dl, Dt, Cl, Ct), D_MIN_KM)
        urban = (URBAN_BUF is not None and site_gdf.geometry.iloc[j].within(URBAN_BUF))
        margin = MARGIN_COV_URBAN if urban else MARGIN_COV_RURAL
        thr = COV_RSRP_DBM_URBAN if urban else COV_RSRP_DBM_RURAL
        ok_any = np.zeros(nD, bool)
        for b in bands:
            rmax = b.get("rmax_km", None)
            mask = (d <= rmax) if rmax else np.ones(nD, bool)
            pl = path_loss(d, b, b["hb_m"]) - b.get("env_gain_db",0.0)
            rx = b["tx_dbm"] - pl - margin
            ok_any |= (mask & (rx >= thr))
        idx = np.where(ok_any)[0]
        if idx.size:
            rows.extend(idx.tolist()); cols.extend([j]*idx.size)
    return csr_matrix((np.ones(len(rows), np.uint8), (rows, cols)), shape=(nD, nC)).tocsr()
```

2.4 Demand Clustering with DBSCAN and HDBSCAN

```python
#Demand hotspot clustering to seed macro candidates (HDBSCAN/DBSCAN) ===
import numpy as np, pandas as pd, geopandas as gpd
from scipy.spatial.distance import cdist

def _to_3857(gdf): return gpd.GeoDataFrame(gdf).to_crs(3857) if gdf.crs is not None else gpd.GeoDataFrame(gdf).set_crs(4326).to_crs(3857)

HOT_FRACTION = 0.30  # top 30% by pop
hot = demand.sort_values("pop", ascending=False).head(max(50, int(HOT_FRACTION*len(demand)))).reset_index(drop=True)

hot_3857 = _to_3857(hot)
X = np.c_[hot_3857.geometry.x.values, hot_3857.geometry.y.values]
w = hot["pop"].to_numpy(float)

labels = None
try:
    import hdbscan
    clusterer = hdbscan.HDBSCAN(min_cluster_size=25, min_samples=10, cluster_selection_epsilon=150.0, metric="euclidean")
    labels = clusterer.fit(X).labels_
except Exception:
    from sklearn.cluster import DBSCAN
    EPS_M, MIN_SAMPLES = 350.0, 12
    labels = DBSCAN(eps=EPS_M, min_samples=MIN_SAMPLES, metric="euclidean").fit(X).labels_

seeds = []
for lab in np.unique(labels):
    if lab < 0:  # noise
        continue
    idx = np.where(labels == lab)[0]
    if idx.size == 0:
        continue
    Xi = X[idx]; wi = w[idx]
    from scipy.spatial.distance import cdist as _cd
    D = _cd(Xi, Xi)
    med = idx[np.argmin((D * wi[None, :]).sum(axis=1))]
    seeds.append(hot.iloc[med].geometry)

seed_gdf = gpd.GeoDataFrame(geometry=seeds, crs=hot.crs).to_crs(4326)
seed_gdf["source"] = "cluster_seed"
if 'existing' not in globals():
    existing = gpd.GeoDataFrame(geometry=[], crs=4326)

cands = pd.concat([existing[["geometry"]], seed_gdf[["geometry"]]], ignore_index=True)          .drop_duplicates(subset="geometry").reset_index(drop=True)
cands = gpd.GeoDataFrame(cands, geometry="geometry", crs=4326)
print(f"[4B] hotspot clusters + seeds: {len(seed_gdf)} | candidates for 10R: {len(cands)}")
```

2.5 Elevation aware Macro ranking

```
#Elevation-aware macro candidate ranking
import numpy as np, pandas as pd, geopandas as gpd, math
from sklearn.neighbors import BallTree

def _sigmoid(x): return 1.0/(1.0+np.exp(-x))

# ---- 1) Population proximity (3 km) using fast haversine BallTree
R_EARTH_KM = 6371.0088
pop_vals = demand["pop"].to_numpy(float)
pts_dem = np.radians(np.c_[demand.geometry.y.values, demand.geometry.x.values])
tree = BallTree(pts_dem, metric="haversine")

near_km = 3.0
indices = []
near_pop = np.zeros(len(cands), float)
query_pts = np.radians(np.c_[cands.geometry.y.values, cands.geometry.x.values])
rad = near_km / R_EARTH_KM
nbrs = tree.query_radius(query_pts, r=rad)
for i, idx in enumerate(nbrs):
    if idx.size:
        near_pop[i] = float(pop_vals[idx].sum())
cands["near_pop_3km"] = near_pop

# ---- 2) Terrain features from DEM: elevation, local relief (~300 m), slope
cands["elev_m"] = np.nan
cands["relief_m"] = np.nan
cands["slope_deg"] = np.nan

try:
    import rasterio
    with rasterio.open(DEM_TIF) as src:
        # SRTM ~30 m → 11x11 ≈ ~330 m window
        WIN = 11
        for i, g in enumerate(cands.geometry.values):
            r, c = src.index(float(g.x), float(g.y))
            r0 = max(0, r - WIN//2); r1 = min(src.height, r + WIN//2 + 1)
            c0 = max(0, c - WIN//2); c1 = min(src.width,  c + WIN//2 + 1)
            arr = src.read(1, window=((r0, r1), (c0, c1)))
            if arr.size == 0: continue
            zc = float(arr[arr.shape[0]//2, arr.shape[1]//2])
            zmed = float(np.nanmedian(arr))
            # slope from simple central differences
            gy, gx = np.gradient(arr.astype("float32"))
            slope = np.degrees(np.arctan(np.hypot(gy, gx)))
            cands.at[i, "elev_m"]   = zc
            cands.at[i, "relief_m"] = zc - zmed
            cands.at[i, "slope_deg"]= float(np.nanmedian(slope))
except Exception as e:
    print("[4B-ELEV] DEM sampling skipped:", e)
```

```python
# ---- 3) Composite terrain score
relief = np.nan_to_num(cands["relief_m"].to_numpy(float), nan=0.0)
_elev_arr = cands["elev_m"].to_numpy(float)
_elev_med = np.nanmedian(_elev_arr) if np.isfinite(_elev_arr).any() else 100.0
elev   = np.nan_to_num(_elev_arr, nan=_elev_med)
slope  = np.nan_to_num(cands["slope_deg"].to_numpy(float),nan=8.0)

relief_score = _sigmoid((relief-10.0)/20.0)              # ~0 at flat/valley, 1 to 1 for +50-60 m relief
elev_cap     = 1.0 - _sigmoid((elev-250.0)/80.0)         # start penalizing > ~250 m
slope_pen    = 1.0 - _sigmoid((slope-12.0)/4.0) * 0.35   # small penalty for very steep ground

terrain_score = np.clip(relief_score * elev_cap * slope_pen, 0.0, 1.0)

# ---- 4) Re-rank candidates (population × terrain)
W_TERRAIN = 0.35
score = cands["near_pop_3km"].to_numpy(float) * (1.0 - W_TERRAIN + W_TERRAIN*terrain_score)
cands["elev_rank_score"] = score

# ---- 5) Thin out by min separation while keeping the highest scores
MIN_SEP_KM = 1.2  # macro min spacing; tweak 0.8-1.5 depending on density
order = np.argsort(-score)
keep_idx = []
if len(order):
    pts = np.c_[cands.geometry.x.values, cands.geometry.y.values]
    for k in order:
        p = pts[k]
        if not keep_idx:
            keep_idx.append(k); continue
        # great-circle distance to kept
        dsep = np.array([
            haversine_km(p[0], p[1], pts[j][0], pts[j][1])
            for j in keep_idx
        ])
        if (dsep >= MIN_SEP_KM).all():
            keep_idx.append(k)

cands = cands.iloc[keep_idx].reset_index(drop=True)

print(f"[4B-ELEV] Elevation-aware candidates kept: {len(cands)} "
      f"(median relief={np.nanmedian(cands['relief_m']):.1f} m, "
      f"median slope={np.nanmedian(cands['slope_deg']):.1f}°)")
```

2.6 Macro coverage calculation

```python
# === 5-Macro Coverage (RSRP) ===
if 'cands' not in globals() or cands is None or len(cands)==0:
    cands = existing[["geometry"]].copy() if len(existing) else gpd.GeoDataFrame(geometry=[], crs=4326)

def _rsrp_A(demand_gdf, cand_gdf, bands, margin_u, margin_r, thr_u, thr_r, rmax_scale=1.0):
    nD, nC = len(demand_gdf), len(cand_gdf)
    if nC == 0: return csr_matrix((nD, 0), dtype=np.uint8)
    Dl, Dt = demand_gdf.geometry.x.values, demand_gdf.geometry.y.values
    rows, cols = [], []
    site_is_urban = np.array([(URBAN_BUF is not None and cand_gdf.geometry.iloc[j].within(URBAN_BUF)) for j in range(nC)])
    site_margin = np.where(site_is_urban, margin_u, margin_r)
    site_thr    = np.where(site_is_urban, thr_u,    thr_r)
    for j in range(nC):
        Cl, Ct = cand_gdf.geometry.iloc[j].x, cand_gdf.geometry.iloc[j].y
        d = np.maximum(haversine_km(Dl, Dt, Cl, Ct), D_MIN_KM)
        ok_any = np.zeros(nD, bool)
        for b in bands:
            rmax = b.get("rmax_km", None)
            mask_r = (d <= rmax * rmax_scale) if rmax is not None else np.ones(nD, bool)
            pl  = path_loss(d, b, b["hb_m"]) - b.get("env_gain_db", 0.0)
            rx  = b["tx_dbm"] - pl - site_margin[j]
            ok_any |= (mask_r & (rx >= site_thr[j]))
        idx = np.where(ok_any)[0]
        if idx.size:
            rows.extend(idx.tolist()); cols.extend([j]*idx.size)
    return csr_matrix((np.ones(len(rows), np.uint8), (rows, cols)), shape=(nD, nC)).tocsr()

def ensure_macro_coverage(demand_gdf, cand_gdf, bands):
    must = demand_gdf["must_serve"].to_numpy(bool)
    A1 = _rsrp_A(demand_gdf, cand_gdf, bands, MARGIN_COV_URBAN, MARGIN_COV_RURAL, COV_RSRP_DBM_URBAN, COV_RSRP_DBM_RURAL, 1.0)
    coverable1 = (A1[must].getnnz(axis=1) > 0) if A1.shape[1]>0 else np.zeros(must.sum(), bool)
    print(f"[Coverage/Diag] Stage1 coverable must-serve: {int(coverable1.sum())}/{int(must.sum())}")
    if coverable1.all():
        return A1, cand_gdf, "strict"
    A2 = _rsrp_A(demand_gdf, cand_gdf, bands, MARGIN_COV_URBAN, MARGIN_COV_RURAL, COV_RSRP_DBM_URBAN-3, COV_RSRP_DBM_RURAL-3, 1.25)
    coverable2 = (A2[must].getnnz(axis=1) > 0) if A2.shape[1]>0 else np.zeros(must.sum(), bool)
    print(f"[Coverage/Diag] Stage2 (-3 dB thr, +25% rmax) coverable: {int(coverable2.sum())}/{int(must.sum())}")
    if coverable2.all():
        return A2, cand_gdf, "relaxed"
    # Synthesize at uncovered must-serve locations
    miss_mask = ~coverable2
    add_idx_global = np.where(must)[0][np.where(miss_mask)[0]]
    synth = demand_gdf.iloc[add_idx_global][["geometry"]].copy()
    synth["source"] = "synth_macro"
    cand2 = pd.concat([cand_gdf, synth], ignore_index=True).drop_duplicates(subset="geometry").reset_index(drop=True)
    A3 = _rsrp_A(demand_gdf, cand2, bands, max(MARGIN_COV_URBAN-4,8), max(MARGIN_COV_RURAL-4,6), COV_RSRP_DBM_URBAN-5, COV_RSRP_DBM_RURAL-5, 1.5)
    coverable3 = (A3[must].getnnz(axis=1) > 0) if A3.shape[1]>0 else np.zeros(must.sum(), bool)
    print(f"[Coverage/Diag] Stage3 (synth + -5 dB thr, +50% rmax) coverable: {int(coverable3.sum())}/{int(must.sum())}")
    if not coverable3.all():
        print("[WARN] Some must-serve hexes remain uncovered - consider tweaking low-band rmax or thresholds.")
    return A3, cand2, "rescue"

A, cands, cov_stage = ensure_macro_coverage(demand, cands, BANDS_MACRO)
print(f"[Macro/RSRP] A: {A.shape}, links={A.nnz:,} | stage='{cov_stage}'")
```

2.7 Filter to get a realistic Macro subset

```python
# === filtering a realistic macro subset (set-cover + min separation) ===
import numpy as np
from scipy.spatial.distance import cdist

MAX_MACROS = globals().get("MAX_MACROS", 60)          # hard cap
MIN_MACRO_SEP_KM = globals().get("MIN_MACRO_SEP_KM", 1.5)  # min spacing between selected macros

assert 'A' in globals() and 'cands' in globals(), "Run 5-Macro Coverage first (A, cands)."

nD, nC = A.shape
coords = np.c_[cands.geometry.x.values, cands.geometry.y.values]
must   = demand["must_serve"].to_numpy(bool)
pop    = demand["pop"].to_numpy(float)

A_csr  = A.tocsr()
covered = np.zeros(nD, dtype=bool)
chosen  = []

# precompute candidate-to-candidate distances
if nC > 0:
    Dcc = cdist(coords, coords, lambda a,b: haversine_km(a[0], a[1], b[0], b[1]))
else:
    Dcc = np.zeros((0,0))

def sep_ok(idx):
    if not chosen:
        return True
    return np.all(Dcc[idx, chosen] >= MIN_MACRO_SEP_KM)

# ---- greedy set-cover over must-serve ----
while nC > 0 and (not covered[must].all()) and len(chosen) < MAX_MACROS:
    need = (~covered) & must
    best, best_gain, best_tiebreak = -1, -1, -1.0

    for j in range(nC):
        if j in chosen:
            continue
        if not sep_ok(j):
            continue
        col = A_csr[:, j].toarray().ravel().astype(bool)
        gain = int((need & col).sum())
        if gain <= 0:
            continue
        pop_gain = float(pop[col].sum())
        if gain > best_gain or (gain == best_gain and pop_gain > best_tiebreak):
            best, best_gain, best_tiebreak = j, gain, pop_gain

    if best < 0:
        break

    chosen.append(best)
    covered |= A_csr[:, best].toarray().ravel().astype(bool)

print(f"[6S] Selected macros: {len(chosen)} | Must-serve covered: {(covered[must].all())} "
      f"| Remaining uncovered: {int((~covered & must).sum())}")
```

```python
if not covered[must].all():
    _orig_sep = MIN_MACRO_SEP_KM
    MIN_MACRO_SEP_KM = max(0.9, _orig_sep - 0.3)  # relax by 300 m once
    covered = np.zeros(nD, dtype=bool)
    chosen  = []
    while nC > 0 and (not covered[must].all()) and len(chosen) < MAX_MACROS:
        need = (~covered) & must
        best, best_gain, best_tiebreak = -1, -1, -1.0
        for j in range(nC):
            if j in chosen:
                continue
            if not sep_ok(j):
                continue
            col = A_csr[:, j].toarray().ravel().astype(bool)
            gain = int((need & col).sum())
            if gain <= 0:
                continue
            pop_gain = float(pop[col].sum())
            if gain > best_gain or (gain == best_gain and pop_gain > best_tiebreak):
                best, best_gain, best_tiebreak = j, gain, pop_gain
        if best < 0:
            break
        chosen.append(best)
        covered |= A_csr[:, best].toarray().ravel().astype(bool)
    print(f"[6S] (relaxed) Selected macros: {len(chosen)} | Must-serve covered: {(covered[must].all())} "
          f"| Remaining uncovered: {int((~covered & must).sum())}")
    MIN_MACRO_SEP_KM = _orig_sep  # restore

# --- building selected_macros and taging existing vs new ---
if chosen:
    selected_macros = cands.iloc[chosen].copy().reset_index(drop=True)
else:
    selected_macros = gpd.GeoDataFrame(columns=["geometry"], crs=4326)

selected_macros["site_type"] = "macro"
selected_macros["origin"] = "new"

if len(selected_macros) and len(existing):
    D = cdist(
        np.c_[selected_macros.geometry.x.values, selected_macros.geometry.y.values],
        np.c_[existing.geometry.x.values, existing.geometry.y.values],
        lambda a,b: haversine_km(a[0], a[1], b[0], b[1])
    )
    near = (D.min(axis=1) <= 0.3) if D.size else np.zeros(len(selected_macros), bool)
    selected_macros.loc[near, "origin"] = "existing"

print(f"[6S] Macros: {len(selected_macros)} (existing={int((selected_macros['origin']=='existing').sum())})")
```

2.8 Enriching the selected Macros for better coverage

```python
# === Enriching Macros: elevation, coverage radius, rough LTE capacity ===
try:
    import rasterio
except Exception:
    rasterio = None

if not len(selected_macros):
    print("[15M] No macros to enrich.")
else:
    elev = np.full(len(selected_macros), np.nan, float)
    if rasterio is not None and os.path.exists(DEM_TIF):
        try:
            with rasterio.open(DEM_TIF) as src:
                coords = [(float(g.x), float(g.y)) for g in selected_macros.geometry.values]
                elev_vals = list(src.sample(coords))
                elev = np.array([v[0] if len(v) else np.nan for v in elev_vals], float)
        except Exception as e:
            print("[15M] DEM sampling skipped:", e)
    selected_macros["elev_m"] = np.round(elev, 1)

    Dl, Dt = demand.geometry.x.values, demand.geometry.y.values
    pop = demand["pop"].to_numpy(float)
    cov_km = np.zeros(len(selected_macros), float)
    cov_pop= np.zeros(len(selected_macros), float)
    for j in range(len(selected_macros)):
        Cl, Ct = float(selected_macros.geometry.iloc[j].x), float(selected_macros.geometry.iloc[j].y)
        d = np.maximum(haversine_km(Dl, Dt, Cl, Ct), D_MIN_KM)
        urban = (URBAN_BUF is not None and selected_macros.geometry.iloc[j].within(URBAN_BUF))
        margin = MARGIN_COV_URBAN if urban else MARGIN_COV_RURAL
        thr    = COV_RSRP_DBM_URBAN if urban else COV_RSRP_DBM_RURAL
        ok_any = np.zeros(len(demand), bool)
        for b in BANDS_MACRO:
            mask_r = (d <= b.get("rmax_km", np.inf))
            pl = path_loss(d, b, b["hb_m"]) - b.get("env_gain_db",0.0)
            rx = b["tx_dbm"] - pl - margin
            ok_any |= (mask_r & (rx >= thr))
        if ok_any.any():
            cov_km[j]  = float(np.quantile(d[ok_any], 0.90))
            cov_pop[j] = float(pop[ok_any].sum())
    selected_macros["coverage_km"]     = np.round(cov_km, 2)
    selected_macros["covered_pop_raw"] = np.round(cov_pop).astype(int)

    def _lte_eff_bphz(snr_db, gap_db=4.5):
        snr_lin = 10**(snr_db/10.0); gap_lin = 10**(gap_db/10.0)
        return 0.75*np.log2(1 + snr_lin/gap_lin)
```

```python
cap_macro = np.zeros(len(selected_macros), float)
for j in range(len(selected_macros)):
    Cl, Ct = float(selected_macros.geometry.iloc[j].x), float(selected_macros.geometry.iloc[j].y)
    d = np.maximum(haversine_km(Dl, Dt, Cl, Ct), D_MIN_KM)
    urban = (URBAN_BUF is not None and selected_macros.geometry.iloc[j].within(URBAN_BUF))
    margin = MARGIN_COV_URBAN if urban else MARGIN_COV_RURAL
    tot = 0.0
    for b in BANDS_MACRO:
        mask_r = (d <= b.get("rmax_km", np.inf))
        pl = path_loss(d, b, b["hb_m"]) - b.get("env_gain_db",0.0)
        rx = b["tx_dbm"] - pl - margin
        thr = COV_RSRP_DBM_URBAN if urban else COV_RSRP_DBM_RURAL
        ok = mask_r & (rx >= thr)
        if ok.any():
            snr = np.minimum(
                snr_db_link(
                    rx[ok],
                    b["bw_mhz"],
                    NF_LTE_DB if b.get("tech","LTE")=="LTE" else NF_NR_DB,
                    IO_HEADROOM_DB_URBAN if urban else IO_HEADROOM_DB_RURAL
                ),
                SNR_CAP_DB
            )

            snr_med = float(np.median(snr))
            eff = _lte_eff_bphz(snr_med)
            tot += SECTORS_PER_SITE * b["bw_mhz"]*1e6 * eff * EFF_OVERHEAD / 1e6
    cap_macro[j] = tot
selected_macros["sectors"]       = SECTORS_PER_SITE
selected_macros["capacity_mbps"] = np.round(cap_macro, 1)
print("Macros enriched.")
```

2.9 BackHaul demand calculation

```
# === BH Demand ===
hex_bh_mbps = (
    demand["pop"].astype(float)
    * MARKET_SHARE * ADOPTION_RATE * CONCURRENCY * BH_TARGET_MBPS_PER_USER
).to_numpy(float)
bh_need_s = pd.Series(hex_bh_mbps, index=demand.index, name="bh_need")
print(f"[12A] BH total need: {hex_bh_mbps.sum():,.0f} Mbps")
```

2.10 Overlay Tuning

```
# === NR Tuning (calibrated, followed all laws of physics ) ===

# 3.5 GHz overlay macros + street-level small cells:
BANDS_NR[0]["rmax_km"] = 2.2      # modest extension; realistic for 100 MHz @ 3.5 GHz
BANDS_NR[0]["snr_db"]  = 3.5      # viability gate; robust CQI at edge
SMALLCELL_PARAMS.update({
    "tx_dbm": max(SMALLCELL_PARAMS.get("tx_dbm", 40), 40),
    "hb_m":   max(SMALLCELL_PARAMS.get("hb_m", 10), 10),
    "rmax_km": 0.95                # light bump to reflect street canyon reach
})

# Busy-hour margins (kept realistic):
MARGIN_CAP_URBAN = 9.5            # was 12-10; now 9.5 to avoid over-pruning
MARGIN_CAP_RURAL = max(9.0, MARGIN_CAP_RURAL)

# Candidate caps and min separations:
MAX_NR_OVERLAYS = 220
MAX_SMALLCELLS  = 200
SC_MIN_SEP_KM   = 0.45

print("[NR tune] rmax_km=", BANDS_NR[0]["rmax_km"],
      "| snr_db=", BANDS_NR[0]["snr_db"],
      "| smallcell rmax_km=", SMALLCELL_PARAMS["rmax_km"],
      "| MARGIN_CAP_URBAN=", MARGIN_CAP_URBAN,
      "| MARGIN_CAP_RURAL=", MARGIN_CAP_RURAL,
      "| SC_MIN_SEP_KM=", SC_MIN_SEP_KM)
```

2.11 Tower candidate selection

```
# === Capacity Candidates ===
band = BANDS_NR[0]

if len(selected_macros):
    Dmh = np.maximum(
        cdist(
            np.c_[demand.geometry.x.values, demand.geometry.y.values],
            np.c_[selected_macros.geometry.x.values, selected_macros.geometry.y.values],
            lambda a,b: haversine_km(a[0], a[1], b[0], b[1])
        ), D_MIN_KM
    )
    bha = bh_need_s.to_numpy()
    scores = []
    for j in range(Dmh.shape[1]):
        rows = np.where(Dmh[:, j] <= band["rmax_km"])[0]
        scores.append(bha[rows].sum() if rows.size else 0.0)

    nr_overlays = selected_macros.copy()[["geometry"]]
    nr_overlays["site_type"] = "nr_overlay"
    nr_overlays["overlay_score"] = scores
    nr_overlays = nr_overlays.sort_values("overlay_score", ascending=False).head(MAX_NR_OVERLAYS)
else:
    nr_overlays = gpd.GeoDataFrame(columns=["geometry","site_type","overlay_score"], crs=4326)

hot = demand.assign(bh_need=bh_need_s).sort_values("bh_need", ascending=False).head(int(MAX_SMALLCELLS*5))
sc = gpd.GeoDataFrame(columns=["geometry","site_type"], crs=4326)
if len(hot):
    pts = np.c_[hot.geometry.x.values, hot.geometry.y.values]
    chosen = []
    for i in range(len(hot)):
        if len(chosen) >= MAX_SMALLCELLS: break
        p = pts[i]
        if not chosen:
            chosen.append(i); continue
        dsep = cdist([p], pts[chosen], lambda a,b: haversine_km(a[0], a[1], b[0], b[1]))[0]
        if np.all(dsep >= SC_MIN_SEP_KM):
            chosen.append(i)
    if chosen:
        sc = hot.iloc[chosen][["geometry"]].copy(); sc["site_type"] = "smallcell"

cap_cands = pd.concat([nr_overlays[["geometry","site_type"]], sc], ignore_index=True)\
            .drop_duplicates(subset="geometry").reset_index(drop=True)
cap_cands = gpd.GeoDataFrame(cap_cands, geometry="geometry", crs=4326)
if "site_type" not in cap_cands.columns: cap_cands["site_type"] = "smallcell"

print(f"[10C] overlays={(cap_cands.site_type=='nr_overlay').sum()}, smallcells={(cap_cands.site_type=='smallcell').sum()}, total={len(cap_cands)}")
```

2.12 Greedy selection of candidates that can handle the required capacity

```python
# === Capacity Greedy (physics-gated; non-zero capacity for NR/SC even with no assignment) ===
def eff_nr_bphz(snr_db, gap_db=3.5):
    snr_lin = 10**(snr_db/10.0); gap_lin = 10**(gap_db/10.0)
    return 0.8*np.log2(1 + snr_lin/gap_lin)

def _site_viable_and_cap(j, Dkm, cap_cands, band, margin_u, margin_r):
    st = str(cap_cands.iloc[j].get("site_type","smallcell"))
    tx = SMALLCELL_PARAMS["tx_dbm"] if st=="smallcell" else band["tx_dbm"]
    hb = SMALLCELL_PARAMS["hb_m"]   if st=="smallcell" else band["hb_m"]
    rmax = SMALLCELL_PARAMS["rmax_km"] if st=="smallcell" else band["rmax_km"]
    snr_gate = band.get("snr_db", 3.5)
    rows = np.where(Dkm[:, j] <= rmax)[0]
    if rows.size == 0:
        return False, 0.0, rows, np.nan
    d = Dkm[rows, j]
    margin = margin_u if (URBAN_BUF is not None and cap_cands.geometry.iloc[j].within(URBAN_BUF)) else margin_r
    pl = path_loss(d, band, hb)
    rx = tx - pl - margin
    snr = np.minimum(
        snr_db_link(
            rx,
            band["bw_mhz"],
            NF_NR_DB,
            IO_HEADRCOM_DB_URBAN if (URBAN_BUF is not None and cap_cands.geometry.iloc[j].within(URBAN_BUF)) else IO_HEADROOM_DB_RURAL
        ),
        SNR_CAP_DB
    )

    ok = snr >= snr_gate
    rows_ok = rows[ok]
    if rows_ok.size == 0:
        return False, 0.0, rows_ok, np.nan
    snr_p50 = float(np.median(snr[ok]))
    eff = eff_nr_bphz(snr_p50)
    cap_est = SECTORS_PER_SITE * band["bw_mhz"]*1e6 * eff * EFF_OVERHEAD / 1e6
    return True, cap_est, rows_ok, snr_p50

if len(cap_cands)==0:
    chosen_cap_idx = []
    print("[11] No capacity candidates — skipping.")
else:
    remaining = (demand["pop"].astype(float) * MARKET_SHARE * ADOPTION_RATE * CONCURRENCY * BH_TARGET_MBPS_PER_USER).to_numpy(float)
    total_need = remaining.sum()
    served = 0.0

    Dkm = np.maximum(
        cdist(
            np.c_[demand.geometry.x.values, demand.geometry.y.values],
            np.c_[cap_cands.geometry.x.values, cap_cands.geometry.y.values],
            lambda a,b: haversine_km(a[0], a[1], b[0], b[1])
        ), D_MIN_KM
    )
    band = BANDS_NR[0]
```

```python
# Pre-compute viability and a non-zero fallback capacity per site
viable = np.zeros(len(cap_cands), bool)
cap_fallback = np.zeros(len(cap_cands), float)
rows_cache = [np.array([], int) for _ in range(len(cap_cands))]
snr_p50_cache = np.full(len(cap_cands), np.nan, float)
for j in range(len(cap_cands)):
    ok, cap_est, rows_ok, snr_p50 = _site_viable_and_cap(j, Dkm, cap_cands, band, MARGIN_CAP_URBAN, MARGIN_CAP_RURAL)
    viable[j] = ok
    cap_fallback[j] = max(0.1, cap_est)  # enforcing tiny positive floor to avoid exact 0 in UI
    rows_cache[j] = rows_ok
    snr_p50_cache[j] = snr_p50

chosen_cap_idx = []
# Greedy pick among viable only options
while True:
    best_j, best_gain = -1, 0.0
    for j in range(len(cap_cands)):
        if j in chosen_cap_idx or not viable[j]:
            continue
        rows = rows_cache[j]
        if rows.size == 0:
            continue
        gain = min(cap_fallback[j], remaining[rows].sum())
        if gain > best_gain:
            best_gain, best_j = gain, j
    if best_j < 0:
        break
    chosen_cap_idx.append(best_j)
    rows = rows_cache[best_j]
    cap_j = cap_fallback[best_j]
    need = remaining[rows]
    if need.sum() > 0:
        frac = min(1.0, cap_j / need.sum())
        take = need * frac
        remaining[rows] = need - take
        served += take.sum()

print(f"[11] chosen NR/small sites={len(chosen_cap_idx)} | BH served={served:.0f}/{total_need:.0f} ({served/max(1,total_need):.1%})")
```

2.13 Recomputing NR/Small-cell per-site metrics after selection

```
# 1) Choosing the source for NR/SC candidates:
if 'cap_cands' in globals() and cap_cands is not None and len(cap_cands):
    nr_cands = cap_cands.copy()
else:
    # Fallback: extract NR overlays + small cells from selected_sites if available
    if 'selected_sites' in globals() and len(selected_sites):
        nr_cands = selected_sites[selected_sites['site_type'] != 'macro'][['geometry','site_type']].copy()
    else:
        raise RuntimeError("No NR/Small-cell candidates found: cap_cands and selected_sites are both unavailable/empty.")

nr_cands = nr_cands.reset_index(drop=True)
if 'site_type' not in nr_cands.columns:
    nr_cands['site_type'] = 'smallcell'

# 2) Prepare demand arrays
if 'demand' not in globals() or not len(demand):
    raise RuntimeError("Missing 'demand' GeoDataFrame.")

Dl = demand.geometry.x.values
Dt = demand.geometry.y.values
pop = demand['pop'].astype(float).values
nD  = len(demand)

# 3) NR band + small-cell params + margins
bnr = BANDS_NR[0]  # assumes at least one NR band
def _site_phys(st):
    """Return (tx_dbm, hb_m, rmax_km, margin_db) for site type."""
    if st == 'smallcell':
        tx = float(SMALLCELL_PARAMS['tx_dbm'])
        hb = float(SMALLCELL_PARAMS['hb_m'])
        rmax = float(SMALLCELL_PARAMS['rmax_km'])
        margin = float(MARGIN_CAP_URBAN)  # cap margins for NR/smallcells
    else:  # 'nr_overlay'
        tx = float(bnr['tx_dbm'])
        hb = float(bnr['hb_m'])
        rmax = float(bnr['rmax_km'])
        margin = float(MARGIN_CAP_URBAN)
    return tx, hb, rmax, margin
```

```python
# 5) Compute per-site metrics and attach to nr_cands
snr_gate = float(bnr.get('snr_db', 6.0))
nC = len(nr_cands)

covered_pop_raw = np.zeros(nC, dtype=float)
snr_p50 = np.full(nC, np.nan, float)
snr_p10 = np.full(nC, np.nan, float)
cap_mbps = np.zeros(nC, dtype=float)

for j in range(nC):
    st = str(nr_cands.loc[j, 'site_type'])
    tx, hb, rmax, margin = _site_phys(st)
    # demand rows within rmax
    rows = np.where(Dkm[:, j] <= rmax)[0]
    if rows.size == 0:
        continue

    d = Dkm[rows, j]
    pl = path_loss(d, bnr, hb)
    rx = tx - pl - margin
    # Use a conservative urban/rural interference headroom
    ioc_here = IO_HEADROOM_DB_URBAN if (URBAN_BUF is not None and nr_cands.geometry.iloc[j].within(URBAN_BUF)) else IO_HEADROOM_DB_RURAL
    snr = np.minimum(
        snr_db_link(
            rx,
            bnr['bw_mhz'],
            NF_NR_DB,
            IO_HEADROOM_DB_URBAN if (URBAN_BUF is not None and nr_cands.geometry.iloc[j].within(URBAN_BUF)) else IO_HEADROOM_DB_RURAL
        ),
        SNR_CAP_DB
    )


    ok = snr >= snr_gate
    if ok.sum() == 0:
        # coverage footprint may exist but no viable SNR
        covered_pop_raw[j] = 0.0
        continue

    # covered pop = sum of pop in cells meeting SNR gate within rmax
    covered_pop_raw[j] = float(pop[rows[ok]].sum())

    # SNR stats
    snr_p50[j] = float(np.percentile(snr[ok], 50))
    snr_p10[j] = float(np.percentile(snr[ok], 10))

    # Capacity estimate (per site): sectors * BW * eff / 1e6
    sectors = int(globals().get('SECTORS_PER_SITE', 3))
    eff = eff_nr_bphz(snr_p50[j]) if np.isfinite(snr_p50[j]) else 0.0
    cap_mbps[j] = sectors * bnr['bw_mhz']*1e6 * eff * float(globals().get('EFF_OVERHEAD', 0.85)) / 1e6

# Attach/overwrite columns
nr_cands['covered_pop_raw'] = covered_pop_raw.astype(int)
nr_cands['snr_p50'] = np.round(snr_p50, 1)
nr_cands['snr_p10'] = np.round(snr_p10, 1)
nr_cands['capacity_mbps'] = np.round(cap_mbps, 1)
```

2.14 Assembling all the selected sites

```
# === Assembling Sites ===
selected_nr = cap_cands.iloc[chosen_cap_idx][["geometry","site_type"]].copy() if len(cap_cands) and len(chosen_cap_idx) else gpd.GeoDataFrame(columns=["geometry","site_type"], crs=4326)

sm = selected_macros.copy()
for col, val in {"elev_m": np.nan, "sectors": SECTORS_PER_SITE, "coverage_km": 0.0, "covered_pop_raw": 0, "capacity_mbps": 0.0}.items():
    if col not in sm.columns: sm[col] = val
if "origin" not in sm.columns: sm["origin"] = "new"
sm = sm[["geometry","site_type","origin","elev_m","sectors","coverage_km","covered_pop_raw","capacity_mbps"]]

selected_sites = pd.concat([sm, selected_nr], ignore_index=True).drop_duplicates(subset="geometry").reset_index(drop=True)
if "site_type" not in selected_sites.columns: selected_sites["site_type"] = "smallcell"
if "origin" not in selected_sites.columns: selected_sites["origin"] = "new"

print(f"[14F] selected_sites: {len(selected_sites)} (macros={(selected_sites.site_type=='macro').sum()}, NR={(selected_sites.site_type!='macro').sum()})")
```

2.15 Elevation data retrieval for selected sites

```
# === Elevation for ALL sites (macros + NR overlays + small cells) ===
if 'selected_sites' in globals() and len(selected_sites):
    if 'elev_m' not in selected_sites.columns:
        selected_sites["elev_m"] = np.nan
    try:
        import rasterio, os
        if os.path.exists(DEM_TIF):
            with rasterio.open(DEM_TIF) as src:
                coords = [(float(g.x), float(g.y)) for g in selected_sites.geometry.values]
                vals = list(src.sample(coords))
                z = np.array([v[0] if (v is not None and len(v)) else np.nan for v in vals], float)
                selected_sites["elev_m"] = np.round(z, 1)
                print("[12B] Elevation filled for all site types.")
        else:
            print("[12B] DEM_TIF not found; kept NaN.")
    except Exception as e:
        print("[12B] DEM sampling failed:", e)
else:
    print("[12B] No selected_sites present.")
```

2.16 Calculating the final Capacity and RF parameters for cells

```python
# === Assigning Capacity + RF Tables (coverage-based SNR for NR/SC; no zero capacity) ===
from scipy.spatial.distance import cdist
import numpy as np
if not len(selected_sites):
    print("[13] No sites.")
else:
    # Distances
    D = np.maximum(
        cdist(
            np.c_[demand.geometry.x.values, demand.geometry.y.values],
            np.c_[selected_sites.geometry.x.values, selected_sites.geometry.y.values],
            lambda a,b: haversine_km(a[0], a[1], b[0], b[1])
        ), D_MIN_KM
    )
    # rmax per site
    rmax_km = []
    for _, s in selected_sites.iterrows():
        if s["site_type"]=="macro":
            rmax_km.append(float(s.get("coverage_km", max(b.get("rmax_km", np.inf) for b in BANDS_MACRO))))
        elif s["site_type"]=="nr_overlay":
            rmax_km.append(float(BANDS_NR[0]["rmax_km"]))
        else:
            rmax_km.append(float(SMALLCELL_PARAMS["rmax_km"]))
    rmax_km = np.array(rmax_km)

    # Coverage mask
    mask_cov = (D <= rmax_km[None, :])

    # Assignment (prefer NR/SC, then macros)
    assign = np.full(len(demand), -1, int)
    cols_nr = np.where(selected_sites["site_type"]!="macro")[0]
    if cols_nr.size:
        Dnr = np.where(mask_cov[:, cols_nr], D[:, cols_nr], np.inf)
        best = np.argmin(Dnr, axis=1)
        has  = np.isfinite(Dnr[np.arange(len(demand)), best])
        assign[has] = cols_nr[best[has]]
    leftover = np.where(assign<0)[0]
    cols_m = np.where(selected_sites["site_type"]=="macro")[0]
    if leftover.size and cols_m.size:
        Dm = np.where(mask_cov[:, cols_m], D[:, cols_m], np.inf)
        bestm = np.argmin(Dm[leftover], axis=1)
        hasm  = np.isfinite(Dm[leftover, bestm])
        assign[leftover[hasm]] = cols_m[bestm[hasm]]

    # Demand requirement
    bh_need_arr = (demand["pop"].astype(float) * MARKET_SHARE * ADOPTION_RATE * CONCURRENCY * BH_TARGET_MBPS_PER_USER).to_numpy(float)
    req = np.zeros(len(selected_sites), float)
    pos = np.where(assign>=0)[0]
    np.add.at(req, assign[pos], bh_need_arr[pos])
    selected_sites["required_mbps"] = np.round(req, 1)

    # RF tables and capacities
    def _rf_table_html(rows, header):
        th = "<tr>"+"".join([f"<th>{h}</th>" for h in header])+"</tr>"
        tb = "".join(["<tr>"+"".join([f"<td>{c}</td>" for c in r])+"</tr>" for r in rows])
        return f"<table border=1 cellspacing=0 cellpadding=3>{th}{tb}</table>"
```

```python
if "capacity_mbps" not in selected_sites.columns:
    selected_sites["capacity_mbps"] = 0.0
cap = selected_sites["capacity_mbps"].astype(float).to_numpy()

Dl, Dt = demand.geometry.x.values, demand.geometry.y.values
rf_tables = []

def _snr_stats_over_mask(j, st):
    b = BANDS_NR[0]
    hb = SMALLCELL_PARAMS["hb_m"] if st=="smallcell" else b["hb_m"]
    tx = SMALLCELL_PARAMS["tx_dbm"] if st=="smallcell" else b["tx_dbm"]
    rmax = SMALLCELL_PARAMS["rmax_km"] if st=="smallcell" else b["rmax_km"]
    urban = (URBAN_BUF is not None and selected_sites.geometry.iloc[j].within(URBAN_BUF))
    margin = MARGIN_CAP_URBAN if urban else MARGIN_CAP_RURAL
    idx = np.where(mask_cov[:, j])[0]
    if idx.size == 0:
        return np.nan, np.nan
    d = np.maximum(haversine_km(Dl[idx], Dt[idx], float(selected_sites.geometry.iloc[j].x), float(selected_sites.geometry.iloc[j].y)), D_MIN_KM)
    pl = path_loss(d, b, hb)
    rx = tx - pl - margin
    ioc_here = IO_HEADROOM_DB_URBAN if urban else IO_HEADROOM_DB_RURAL
    snr = np.minimum(
        snr_db_link(
            rx,
            b["bw_mhz"],
            NF_NR_DB,
            IO_HEADROOM_DB_URBAN if urban else IO_HEADROOM_DB_RURAL
        ),
        SNR_CAP_DB
    )


    return float(np.nanpercentile(snr, 50)), float(np.nanpercentile(snr, 10))
```

```python
for j, s in selected_sites.iterrows():
    st = s["site_type"]
    rows_html = []
    if st == "macro":
        rows = np.where(assign==j)[0]
        d = np.maximum(haversine_km(Dl[rows], Dt[rows], float(s.geometry.x), float(s.geometry.y)), D_MIN_KM) if rows.size else np.array([])
        urban = (URBAN_BUF is not None and s.geometry.within(URBAN_BUF))
        margin = MARGIN_COV_URBAN if urban else MARGIN_COV_RURAL
        for b in BANDS_MACRO:
            if rows.size:
                mask_r = (d <= b.get("rmax_km", np.inf))
                pl = path_loss(d, b, b["hb_m"]) - b.get("env_gain_db",0.0)
                rx = b["tx_dbm"] - pl - margin
                snr = np.minimum(
                    snr_db_link(
                        rx,
                        b["bw_mhz"],
                        NF_LTE_DB if b.get("tech","LTE")=="LTE" else NF_NR_DB,
                        IO_HEADROOM_DB_URBAN if urban else IO_HEADROOM_DB_RURAL
                    ),
                    SNR_CAP_DB
                )


                v = snr[mask_r]
                p50 = np.percentile(v, 50) if v.size else np.nan
                p10 = np.percentile(v, 10) if v.size else np.nan
            else:
                p50 = p10 = np.nan
            rows_html.append([b["name"], f'{b["bw_mhz"]} MHz', f'{b["tx_dbm"]} dBm', f'{b["hb_m"]} m',
                              f'{b.get("rmax_km",0):.1f} km', f'{p50:.1f} dB', f'{p10:.1f} dB'])
    else:
        b = BANDS_NR[0]
        p50, p10 = _snr_stats_over_mask(j, st)
        if (not np.isfinite(cap[j])) or (cap[j] <= 0.0):
            snr_for_cap = p50 if np.isfinite(p50) else b.get("snr_db", 3.5)
            eff = eff_nr_bphz(np.nan_to_num(snr_for_cap, nan=b.get("snr_db", 3.5)))
            cap[j] = max(cap[j], SECTORS_PER_SITE * b["bw_mhz"]*1e6 * eff * EFF_OVERHEAD / 1e6)  # Mbps
        hb = SMALLCELL_PARAMS["hb_m"] if st=="smallcell" else b["hb_m"]
        tx = SMALLCELL_PARAMS["tx_dbm"] if st=="smallcell" else b["tx_dbm"]
        rmax= SMALLCELL_PARAMS["rmax_km"] if st=="smallcell" else b["rmax_km"]
        rows_html.append([b["name"], f'{b["bw_mhz"]} MHz', f'{tx} dBm', f'{hb} m',
                          f'{rmax:.1f} km', f'{p50:.1f} dB', f'{p10:.1f} dB'])

    rf_tables.append(_rf_table_html(rows_html, ["Band","BW","Tx","Hb","rmax","SNR p50","SNR p10"]))

selected_sites["capacity_mbps"] = np.round(cap, 1)
util = np.divide(selected_sites["required_mbps"].astype(float).to_numpy(),
                 np.maximum(selected_sites["capacity_mbps"].astype(float).to_numpy(), 1e-6))
selected_sites["utilization"] = util
selected_sites["rf_table_html"] = rf_tables
```

```python
# Backfill per-site pop and empirical radius from assignment
pop = demand["pop"].to_numpy(float)
covered_pop = np.zeros(len(selected_sites), int)
cov_km = np.zeros(len(selected_sites), float)
for j in range(len(selected_sites)):
    rows = np.where(assign==j)[0]
    covered_pop[j] = int(pop[rows].sum()) if rows.size else 0
    if rows.size:
        d = D[rows, j]; d = d[np.isfinite(d)]
        cov_km[j] = float(np.quantile(d, 0.90)) if d.size else rmax_km[j]
    else:
        cov_km[j] = rmax_km[j]
selected_sites["covered_pop_raw"] = covered_pop
selected_sites["coverage_km"] = np.round(cov_km, 2)

print(f"[13] RF tables built. Overloaded: {(selected_sites['utilization']>1.0).sum()} / {len(selected_sites)}")
```

2.17 Backfill coverage/population calculation

```python
# === Backfill coverage/pop metrics for ALL site types (drop right after Cell 13) ===
import numpy as np

# Make sure these columns exist
for col, default in {
    "covered_pop_raw": 0,
    "coverage_km": np.nan,
}.items():
    if col not in selected_sites.columns:
        selected_sites[col] = default

Dl, Dt = demand.geometry.x.values, demand.geometry.y.values
pop = demand["pop"].to_numpy(float)

# Helpers to get rmax/hb/tx per site
def _site_phys(st):
    if st == "macro":
        rmax = max(b.get("rmax_km", np.inf) for b in BANDS_MACRO)
        hb   = max(b["hb_m"] for b in BANDS_MACRO)
        tx   = "/".join([str(b["tx_dbm"]) for b in BANDS_MACRO])
    elif st == "nr_overlay":
        rmax = float(BANDS_NR[0]["rmax_km"]); hb = BANDS_NR[0]["hb_m"]; tx = str(BANDS_NR[0]["tx_dbm"])
    else:  # smallcell
        rmax = float(SMALLCELL_PARAMS["rmax_km"]); hb = SMALLCELL_PARAMS["hb_m"]; tx = str(SMALLCELL_PARAMS["tx_dbm"])
    return rmax, hb, tx

# Compute per-site covered population (from assigned hexes) and empirical radius
for j, s in selected_sites.iterrows():
    rows = np.where(assign == j)[0]
    # Covered pop = sum of assigned hex pop (capacity-limited assignment already done)
    selected_sites.at[j, "covered_pop_raw"] = int(pop[rows].sum()) if rows.size else 0

    # Empirical 90th percentile distance of assigned hexes (fallback to rmax if none)
    st = s.get("site_type", "smallcell")
    rmax, hb, tx = _site_phys(st)
    if rows.size:
        d = D[rows, j]
        # ignore inf (shouldn't appear for assigned), guard NaN
        d = d[np.isfinite(d)]
        selected_sites.at[j, "coverage_km"] = float(np.quantile(d, 0.90)) if d.size else rmax
    else:
        selected_sites.at[j, "coverage_km"] = rmax

# Optional: if you want the map popups to always have numbers, coerce NaNs safely
selected_sites["coverage_km"] = selected_sites["coverage_km"].fillna(
    selected_sites["site_type"].map(
        {"macro": max(b.get("rmax_km", 10.0) for b in BANDS_MACRO),
         "nr_overlay": BANDS_NR[0]["rmax_km"]}
    ).fillna(SMALLCELL_PARAMS["rmax_km"])
)
selected_sites["covered_pop_raw"] = selected_sites["covered_pop_raw"].fillna(0).astype(int)
```